

Table 2.3 Special functions and constants

This table summarizes the mathematical notation that we use for functions and constants that arise in formulas describing the performance of algorithms. The formulas for the approximate values extend to provide much more accuracy, if desired (*see reference section*).

function	name	typical value	approximation
$\lfloor x \rfloor$	floor function	$\lfloor 3.14 \rfloor = 3$	x
$\lceil x \rceil$	ceiling function	$\lceil 3.14 \rceil = 4$	x
$\lg N$	binary logarithm	$\lg 1024 = 10$	$1.44 \ln N$
F_N	Fibonacci numbers	$F_{10} = 55$	$\phi^N / \sqrt{5}$
H_N	harmonic numbers	$H_{10} \approx 2.9$	$\ln N + \gamma$
$N!$	factorial function	$10! = 3628800$	$(N/e)^N$
$\lg(N!)$		$\lg(100!) \approx 520$	$N \lg N - 1.44N$
		$e = 2.71828 \dots$	
		$\gamma = 0.57721 \dots$	
		$\phi = (1 + \sqrt{5})/2 = 1.61803 \dots$	
		$\ln 2 = 0.693147 \dots$	
		$\lg e = 1/\ln 2 = 1.44269 \dots$	



Figure 2.2
Harmonic numbers

The harmonic numbers are an approximation to the area under the curve $y = 1/x$. The constant γ accounts for the difference between H_N and $\ln N = \int_1^N dx/x$.

The natural logarithm $\ln N$ is the area under the curve $1/x$ between 1 and N ; the harmonic number H_N is the area under the step function that we define by evaluating $1/x$ at the integers between 1 and N . This relationship is illustrated in Figure 2.2. The formula

$$H_N \approx \ln N + \gamma + 1/(12N),$$

where $\gamma = 0.57721 \dots$ (this constant is known as *Euler’s constant*) gives an excellent approximation to H_N . By contrast with $\lceil \lg N \rceil$ and $\lfloor \lg N \rfloor$, it is better to use the library `log` function to compute H_N than to do so directly from the definition.

The sequence of numbers

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 ...

that are defined by the formula

$$F_N = F_{N-1} + F_{N-2}, \quad \text{for } N \geq 2 \text{ with } F_0 = 0 \text{ and } F_1 = 1$$

are known as the *Fibonacci numbers*, and they have many interesting properties. For example, the ratio of two successive terms approaches the *golden ratio* $\phi = (1 + \sqrt{5})/2 \approx 1.61803 \dots$. More detailed analysis shows that F_N is $\phi^N / \sqrt{5}$ rounded to the nearest integer.

We also have occasion to manipulate the familiar *factorial* function $N!$. Like the exponential function, the factorial arises in the brute-force solution to problems and grows much too fast for such solutions to be of practical interest. It also arises in the analysis of algorithms because it represents all the ways to arrange N objects. To approximate $N!$, we use *Stirling's formula*:

$$\lg N! \approx N \lg N - N \lg e + \lg \sqrt{2\pi N}.$$

For example, Stirling's formula tells us that the number of bits in the binary representation of $N!$ is about $N \lg N$.

Most of the formulas that we consider in this book are expressed in terms of the few functions that we have described in this section. Many other special functions can arise in the analysis of algorithms. For example, the classical *binomial distribution* and related *Poisson approximation* play an important role in the design and analysis of some of the fundamental search algorithms that we consider in Chapters 14 and 15. We discuss functions not listed here when we encounter them.

Exercises

- ▷ 2.5 For what values of N is $10N \lg N > 2N^2$?
- ▷ 2.6 For what values of N is $N^{3/2}$ between $N(\lg N)^2/2$ and $2N(\lg N)^2$?
- 2.7 For what values of N is $2NH_N - N < N \lg N + 10N$?
- 2.8 What is the smallest value of N for which $\log_{10} \log_{10} N > 8$?
- 2.9 Prove that $\lfloor \lg N \rfloor + 1$ is the number of bits required to represent N in binary.
- 2.10 Add columns to Table 2.1 for $N(\lg N)^2$ and $N^{3/2}$.
- 2.11 Add rows to Table 2.1 for 10^7 and 10^8 instructions per second.
- 2.12 Write a C function that computes H_N , using the `log` function from the standard math library.

- 2.13 Write an efficient C function that computes $\lceil \lg \lg N \rceil$. Do not use a library function.
- 2.14 How many digits are there in the decimal representation of 1 million factorial?
- 2.15 How many bits are there in the binary representation of $\lg(N!)$?
- 2.16 How many bits are there in the binary representation of H_N ?
- 2.17 Give a simple expression for $\lfloor \lg F_N \rfloor$.
- 2.18 Give the smallest values of N for which $\lfloor H_N \rfloor = i$ for $1 \leq i \leq 10$.
- 2.19 Give the largest value of N for which you can solve a problem that requires at least $f(N)$ instructions on a machine that can execute 10^9 instructions per second, for the following functions $f(N)$: $N^{3/2}$, $N^{5/4}$, $2NH_N$, $N \lg N \lg \lg N$, and $N^2 \lg N$.

2.4 Big-Oh Notation

The mathematical artifact that allows us to suppress detail when we are analyzing algorithms is called the *O-notation*, or “big-Oh notation,” which is defined as follows.

Definition 2.1 A function $g(N)$ is said to be $O(f(N))$ if there exist constants c_0 and N_0 such that $g(N) < c_0 f(N)$ for all $N > N_0$.

We use the *O-notation* for three distinct purposes:

- To bound the error that we make when we ignore small terms in mathematical formulas
- To bound the error that we make when we ignore parts of a program that contribute a small amount to the total being analyzed
- To allow us to classify algorithms according to upper bounds on their total running times

We consider the third use in Section 2.7, and discuss briefly the other two here.

The constants c_0 and N_0 implicit in the *O-notation* often hide implementation details that are important in practice. Obviously, saying that an algorithm has running time $O(f(N))$ says nothing about the running time if N happens to be less than N_0 , and c_0 might be hiding a large amount of overhead designed to avoid a bad worst case. We would prefer an algorithm using N^2 nanoseconds over one using $\log N$ centuries, but we could not make this choice on the basis of the *O-notation*.

Often, the results of a mathematical analysis are not exact, but rather are approximate in a precise technical sense: The result might be an expression consisting of a sequence of decreasing terms. Just as we are most concerned with the inner loop of a program, we are most concerned with the *leading terms* (the largest terms) of a mathematical expression. The O -notation allows us to keep track of the leading terms while ignoring smaller terms when manipulating approximate mathematical expressions, and ultimately allows us to make concise statements that give accurate approximations to the quantities that we analyze.

Some of the basic manipulations that we use when working with expressions containing the O -notation are the subject of Exercises 2.20 through 2.25. Many of these manipulations are intuitive, but mathematically inclined readers may be interested in working Exercise 2.21 to prove the validity of the basic operations from the definition. Essentially, these exercises say that we can expand algebraic expressions using the O -notation as though the O were not there, then can drop all but the largest term. For example, if we expand the expression

$$(N + O(1))(N + O(\log N) + O(1)),$$

we get six terms

$$N^2 + O(N) + O(N \log N) + O(\log N) + O(N) + O(1),$$

but can drop all but the largest O -term, leaving the approximation

$$N^2 + O(N \log N).$$

That is, N^2 is a good approximation to this expression when N is large. These manipulations are intuitive, but the O -notation allows us to express them mathematically with rigor and precision. We refer to a formula with one O -term as an *asymptotic expression*.

For a more relevant example, suppose that (after some mathematical analysis) we determine that a particular algorithm has an inner loop that is iterated $2NH_N$ times on the average, an outer section that is iterated N times, and some initialization code that is executed once. Suppose further that we determine (after careful scrutiny of the implementation) that each iteration of the inner loop requires a_0 nanoseconds, the outer section requires a_1 nanoseconds, and the initialization part a_2 nanoseconds. Then we know that the average running time of

the program (in nanoseconds) is

$$2a_0NH_N + a_1N + a_2.$$

But it is also true that the running time is

$$2a_0NH_N + O(N).$$

This simpler form is significant because it says that, for large N , we may not need to find the values of a_1 or a_2 to approximate the running time. In general, there could well be many other terms in the mathematical expression for the exact running time, some of which may be difficult to analyze. The O -notation provides us with a way to get an approximate answer for large N without bothering with such terms.

Continuing this example, we also can use the O -notation to express running time in terms of a familiar function, $\ln N$. In terms of the O -notation, the approximation in Table 2.3 is expressed as $H_N = \ln N + O(1)$. Thus, $2a_0N \ln N + O(N)$ is an asymptotic expression for the total running time of our algorithm. We expect the running time to be close to the easily computed value $2a_0N \ln N$ for large N . The constant factor a_0 depends on the time taken by the instructions in the inner loop.

Furthermore, we do not need to know the value of a_0 to predict that the running time for input of size $2N$ will be about twice the running time for input of size N for huge N because

$$\frac{2a_0(2N) \ln(2N) + O(2N)}{2a_0N \ln N + O(N)} = \frac{2 \ln(2N) + O(1)}{\ln N + O(1)} = 2 + O\left(\frac{1}{\log N}\right).$$

That is, the asymptotic formula allows us to make accurate predictions without concerning ourselves with details of either the implementation or the analysis. Note that such a prediction would *not* be possible if we were to have only an O -approximation for the leading term.

The kind of reasoning just outlined allows us to focus on the leading term when comparing or trying to predict the running times of algorithms. We are so often in the position of counting the number of times that fixed-cost operations are performed and wanting to use the leading term to estimate the result that we normally keep track of *only* the leading term, assuming implicitly that a precise analysis like the one just given could be performed, if necessary.

When a function $f(N)$ is asymptotically large compared to another function $g(N)$ (that is, $g(N)/f(N) \rightarrow 0$ as $N \rightarrow \infty$), we some-

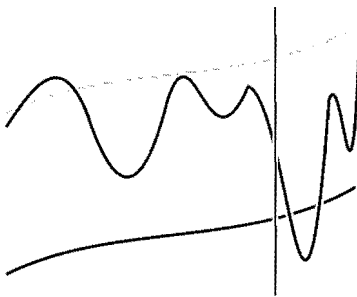


Figure 2.3
Bounding a function with an O -approximation

*In this schematic diagram, the oscillating curve represents a function, $g(N)$, which we are trying to approximate; the black smooth curve represents another function, $f(N)$, which we are trying to use for the approximation; and the gray smooth curve represents $cf(N)$ for some unspecified constant c . The vertical line represents a value N_0 , indicating that the approximation is to hold for $N > N_0$. When we say that $g(N) = O(f(N))$, we expect only that the value of $g(N)$ falls below *some* curve the shape of $f(N)$ to the right of *some* vertical line. The behavior of $f(N)$ could otherwise be erratic (for example, it need not even be continuous).*

times use in this book the (decidedly nontechnical) terminology *about* $f(N)$ to mean $f(N) + O(g(N))$. What we seem to lose in mathematical precision we gain in clarity, for we are more interested in the performance of algorithms than in mathematical details. In such cases, we can rest assured that, for large N (if not for all N), the quantity in question will be close to $f(N)$. For example, even if we know that a quantity is $N(N-1)/2$, we may refer to it as being about $N^2/2$. This way of expressing the result is more quickly understood than the more detailed exact result, and, for example, deviates from the truth only by 0.1 percent for $N = 1000$. The precision lost in such cases pales by comparison with the precision lost in the more common usage $O(f(N))$. Our goal is to be both precise and concise when describing the performance of algorithms.

In a similar vein, we sometimes say that the running time of an algorithm *is proportional to* $f(N)$ when we can prove that it is equal to $cf(N) + g(N)$ with $g(N)$ asymptotically smaller than $f(N)$. When this kind of bound holds, we can project the running time for, say, $2N$ from our observed running time for N , as in the example just discussed. Figure 2.5 gives the factors that we can use for such projection for functions that commonly arise in the analysis of algorithms. Coupled with empirical studies (see Section 2.1), this approach frees us from the task of determining implementation-dependent constants in detail. Or, working backward, we often can easily develop an hypothesis about the functional growth of the running time of a program by determining the effect of doubling N on running time.

The distinctions among O -bounds, *is proportional to*, and *about* are illustrated in Figures 2.3 and 2.4. We use O -notation primarily to learn the fundamental asymptotic behavior of an algorithm; *is proportional to* when we want to predict performance by extrapolation from empirical studies; and *about* when we want to compare performance or to make absolute performance predictions.

Exercises

▷ 2.20 Prove that $O(1)$ is the same as $O(2)$.

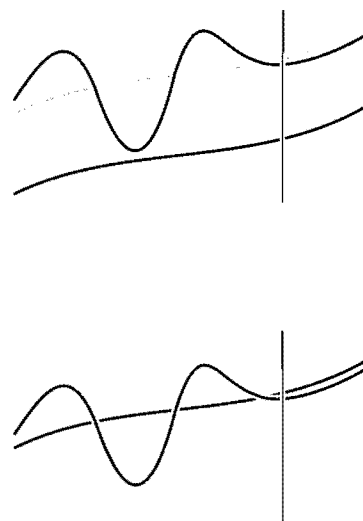


Figure 2.4
Functional approximations

When we say that $g(N)$ is *proportional to* $f(N)$ (top), we expect that it eventually grows like $f(N)$ does, but perhaps offset by an unknown constant. Given some value of $g(N)$, this knowledge allows us to estimate it for larger N . When we say that $g(N)$ is *about* $f(N)$ (bottom), we expect that we can eventually use f to estimate the value of g accurately.

2.21 Prove that we can make any of the following transformations in an expression that uses the O -notation:

$$\begin{aligned} f(N) &\rightarrow O(f(N)), \\ cO(f(N)) &\rightarrow O(f(N)), \\ O(cf(N)) &\rightarrow O(f(N)), \\ f(N) - g(N) = O(h(N)) &\rightarrow f(N) = g(N) + O(h(N)), \\ O(f(N))O(g(N)) &\rightarrow O(f(N)g(N)), \\ O(f(N)) + O(g(N)) &\rightarrow O(g(N)) \quad \text{if } f(N) = O(g(N)). \end{aligned}$$

○ **2.22** Show that $(N + 1)(H_N + O(1)) = N \ln N + O(N)$.

2.23 Show that $N \ln N = O(N^{3/2})$.

● **2.24** Show that $N^M = O(\alpha^N)$ for any M and any constant $\alpha > 1$.

● **2.25** Prove that

$$\frac{N}{N + O(1)} = 1 + O\left(\frac{1}{N}\right).$$

2.26 Suppose that $H_k = N$. Give an approximate formula that expresses k as a function of N .

● **2.27** Suppose that $\lg(k!) = N$. Give an approximate formula that expresses k as a function of N .

○ **2.28** You are given the information that the running time of one algorithm is $O(N \log N)$ and that the running time of another algorithm is $O(N^3)$. What does this statement imply about the relative performance of the algorithms?

○ **2.29** You are given the information that the running time of one algorithm is always about $N \log N$ and that the running time of another algorithm is $O(N^3)$. What does this statement imply about the relative performance of the algorithms?

○ **2.30** You are given the information that the running time of one algorithm is always about $N \log N$ and that the running time of another algorithm is always about N^3 . What does this statement imply about the relative performance of the algorithms?

○ **2.31** You are given the information that the running time of one algorithm is always proportional to $N \log N$ and that the running time of another algorithm is always proportional to N^3 . What does this statement imply about the relative performance of the algorithms?

○ **2.32** Derive the factors given in Figure 2.5: For each function $f(N)$ that appears on the left, find an asymptotic formula for $f(2N)/f(N)$.

1	none
$\lg N$	slight increase
N	double
$N \lg N$	slightly more than double
$N^{3/2}$	factor of $2\sqrt{2}$
N^2	factor of 4
N^3	factor of 8
2^N	square

Figure 2.5
Effect of doubling problem size on running time

Predicting the effect of doubling the problem size on the running time is a simple task when the running time is proportional to certain simple functions, as indicated in this table. In theory, we cannot depend on this effect unless N is huge, but this method is surprisingly effective. Conversely, a quick method for determining the functional growth of the running time of a program is to run that program empirically, doubling the input size for N as large as possible, then work backward from this table.

2.5 Basic Recurrences

As we shall see throughout the book, a great many algorithms are based on the principle of recursively decomposing a large problem into one or more smaller ones, using solutions to the subproblems to solve the original problem. We discuss this topic in detail in Chapter 5, primarily from a practical point of view, concentrating on implementations and applications. We also consider an example in detail in Section 2.6. In this section, we look at basic methods for analyzing such algorithms and derive solutions to a few standard formulas that arise in the analysis of many of the algorithms that we will be studying. Understanding the mathematical properties of the formulas in this section will give us insight into the performance properties of algorithms throughout the book.

Formula 2.1 This formula arises for a program that loops through the input to eliminate one item:

$$C_N = C_{N-1} + N, \quad \text{for } N \geq 2 \text{ with } C_1 = 1.$$

Solution: C_N is about $N^2/2$. To find the value of C_N , we *telescope* the equation by applying it to itself, as follows:

$$\begin{aligned} C_N &= C_{N-1} + N \\ &= C_{N-2} + (N-1) + N \\ &= C_{N-3} + (N-2) + (N-1) + N \\ &\vdots \\ &= C_1 + 2 + \cdots + (N-2) + (N-1) + N \\ &= 1 + 2 + \cdots + (N-2) + (N-1) + N \\ &= \frac{N(N+1)}{2}. \end{aligned}$$

Evaluating the sum $1 + 2 + \cdots + (N-2) + (N-1) + N$ is elementary: The given result follows when we add the sum to itself, but in reverse order, term by term. This result—twice the value sought—consists of N terms, each of which sums to $N+1$.

This simple example illustrates the basic scheme that we use in this section as we consider a number of formulas, which are all based on the principle that recursive decomposition in an algorithm is directly reflected in its analysis. For example, the running time of such

algorithms is determined by the size and number of the subproblems and the time required for the decomposition. Mathematically, the dependence of the running time of an algorithm for an input of size N on its running time for smaller inputs is captured easily with formulas called *recurrence relations*. Such formulas describe precisely the performance of the corresponding algorithms: To derive the running time, we solve the recurrences. More rigorous arguments related to specific algorithms will come up when we get to the algorithms—here, we concentrate on the formulas themselves.

Formula 2.2 This recurrence arises for a recursive program that halves the input in one step:

$$C_N = C_{N/2} + 1, \quad \text{for } N \geq 2 \text{ with } C_1 = 1.$$

Solution: C_N is about $\lg N$. As written, this equation is meaningless unless N is even or we assume that $N/2$ is an integer division. For the moment, we assume that $N = 2^n$, so the recurrence is always well-defined. (Note that $n = \lg N$.) But then the recurrence telescopes even more easily than our first recurrence:

$$\begin{aligned} C_{2^n} &= C_{2^{n-1}} + 1 \\ &= C_{2^{n-2}} + 1 + 1 \\ &= C_{2^{n-3}} + 3 \\ &\vdots \\ &= C_{2^0} + n \\ &= n + 1. \end{aligned}$$

The precise solution for general N depends on the interpretation of $N/2$. In the case that $N/2$ represents $\lfloor N/2 \rfloor$, we have a simple solution: C_N is the number of bits in the binary representation of N , and that number is $\lfloor \lg N \rfloor + 1$, by definition. This conclusion follows immediately from the fact that the operation of eliminating the rightmost bit of the binary representation of any integer $N > 0$ converts it into $\lfloor N/2 \rfloor$ (see Figure 2.6).

Formula 2.3 This recurrence arises for a recursive program that halves the input, but perhaps must examine every item in the input.

$$C_N = C_{N/2} + N, \quad \text{for } N \geq 2 \text{ with } C_1 = 0.$$

N	$(N)_2$	$\lfloor \lg N \rfloor + 1$
1	1	1
2	10	2
3	11	2
4	100	3
5	101	3
6	110	3
7	111	3
8	1000	4
9	1001	4
10	1010	4
11	1011	4
12	1100	4
13	1101	4
14	1110	4
15	1111	4

Figure 2.6
Integer functions and binary representations

Given the binary representation of a number N (center), we obtain $\lfloor N/2 \rfloor$ by removing the rightmost bit. That is, the number of bits in the binary representation of N is 1 greater than the number of bits in the binary representation of $\lfloor N/2 \rfloor$. Therefore, $\lfloor \lg N \rfloor + 1$, the number of bits in the binary representation of N , is the solution to Formula 2.2 for the case that $N/2$ is interpreted as $\lfloor N/2 \rfloor$.

Solution: C_N is about $2N$. The recurrence telescopes to the sum $N + N/2 + N/4 + N/8 + \dots$. (Like Formula 2.2, the recurrence is precisely defined only when N is a power of 2). If the sequence is infinite, this simple geometric sum evaluates to exactly $2N$. Because we use integer division and stop at 1, this value is an approximation to the exact answer. The precise solution involves properties of the binary representation of N .

Formula 2.4 This recurrence arises for a recursive program that has to make a linear pass through the input, before, during, or after splitting that input into two halves:

$$C_N = 2C_{N/2} + N, \quad \text{for } N \geq 2 \text{ with } C_1 = 0.$$

Solution: C_N is about $N \lg N$. This solution is the most widely cited of those we are considering here, because the recurrence applies to a family of standard divide-and-conquer algorithms.

$$\begin{aligned} C_{2^n} &= 2C_{2^{n-1}} + 2^n \\ \frac{C_{2^n}}{2^n} &= \frac{C_{2^{n-1}}}{2^{n-1}} + 1 \\ &= \frac{C_{2^{n-2}}}{2^{n-2}} + 1 + 1 \\ &\vdots \\ &= n. \end{aligned}$$

We develop the solution very much as we did in Formula 2.2, but with the additional trick of dividing both sides of the recurrence by 2^n at the second step to make the recurrence telescope.

Formula 2.5 This recurrence arises for a recursive program that splits the input into two halves and then does a constant amount of other work (see Chapter 5).

$$C_N = 2C_{N/2} + 1, \quad \text{for } N \geq 2 \text{ with } C_1 = 1.$$

Solution: C_N is about $2N$. We can derive this solution in the same manner as we did the solution to Formula 2.4.

We can solve minor variants of these formulas, involving different initial conditions or slight differences in the additive term, using the same solution techniques, although we need to be aware that some recurrences that seem similar to these may actually be rather difficult

to solve. There is a variety of advanced general techniques for dealing with such equations with mathematical rigor (*see reference section*). We will encounter a few more complicated recurrences in later chapters, but we defer discussion of their solution until they arise.

Exercises

▷ **2.33** Give a table of the values of C_N in Formula 2.2 for $1 \leq N \leq 32$, interpreting $N/2$ to mean $\lfloor N/2 \rfloor$.

▷ **2.34** Answer Exercise 2.33, but interpret $N/2$ to mean $\lceil N/2 \rceil$.

▷ **2.35** Answer Exercise 2.34 for Formula 2.3.

○ **2.36** Suppose that f_N is proportional to a constant and that

$$C_N = C_{N/2} + f_N, \quad \text{for } N \geq t \text{ with } 0 \leq C_N < c \text{ for } N < t,$$

where c and t are both constants. Show that C_N is proportional to $\lg N$.

● **2.37** State and prove generalized versions of Formulas 2.3 through 2.5 that are analogous to the generalized version of Formula 2.2 in Exercise 2.36.

2.38 Give a table of the values of C_N in Formula 2.4 for $1 \leq N \leq 32$, for the following three cases: (i) interpret $N/2$ to mean $\lfloor N/2 \rfloor$; (ii) interpret $N/2$ to mean $\lceil N/2 \rceil$; (iii) interpret $2C_{N/2}$ to mean $C_{\lfloor N/2 \rfloor} + C_{\lceil N/2 \rceil}$.

2.39 Solve Formula 2.4 for the case when $N/2$ is interpreted as $\lfloor N/2 \rfloor$, by using a correspondence to the binary representation of N , as in the proof of Formula 2.2. *Hint:* Consider all the numbers less than N .

2.40 Solve the recurrence

$$C_N = C_{N/2} + N^2, \quad \text{for } N \geq 2 \text{ with } C_1 = 0,$$

when N is a power of 2.

2.41 Solve the recurrence

$$C_N = C_{N/\alpha} + 1, \quad \text{for } N \geq 2 \text{ with } C_1 = 0,$$

when N is a power of α .

○ **2.42** Solve the recurrence

$$C_N = \alpha C_{N/2}, \quad \text{for } N \geq 2 \text{ with } C_1 = 1,$$

when N is a power of 2.

○ **2.43** Solve the recurrence

$$C_N = (C_{N/2})^2, \quad \text{for } N \geq 2 \text{ with } C_1 = 1,$$

when N is a power of 2.

- **2.44** Solve the recurrence

$$C_N = \left(2 + \frac{1}{\lg N}\right)C_{N/2}, \quad \text{for } N \geq 2 \text{ with } C_1 = 1,$$

when N is a power of 2.

- **2.45** Consider the family of recurrences like Formula 2.1, where we allow $N/2$ to be interpreted as $\lfloor N/2 \rfloor$ or $\lceil N/2 \rceil$, and we require only that the recurrence hold for $N > c_0$ with $C_N = O(1)$ for $N \leq c_0$. Prove that $\lg N + O(1)$ is the solution to all such recurrences.
- **2.46** Develop generalized recurrences and solutions similar to Exercise 2.45 for Formulas 2.2 through 2.5.

2.6 Examples of Algorithm Analysis

Armed with the tools outlined in the previous three sections, we now consider the analysis of *sequential search* and *binary search*, two basic algorithms for determining whether or not any of a sequence of objects appears among a set of previously stored objects. Our purpose is to illustrate the manner in which we will compare algorithms, rather than to describe these particular algorithms in detail. For simplicity, we assume here that the objects in question are integers. We will consider more general applications in great detail in Chapters 12 through 16. The simple versions of the algorithms that we consider here not only expose many aspects of the algorithm design and analysis problem, but also have many direct applications.

For example, we might imagine a credit-card company that has N credit risks or stolen credit cards, and that wants to check whether any of M given transactions involves any one of the N bad numbers. To be concrete, we might think of N being large (say on the order of 10^3 to 10^6) and M being huge (say on the order of 10^6 to 10^9) for this application. The goal of the analysis is to be able to estimate the running times of the algorithms when the values of the parameters fall within these ranges.

Program 2.1 implements a straightforward solution to the search problem. It is packaged as a C function that operates on an array (see Chapter 3) for better compatibility with other code that we will examine for the same problem in Part 4, but it is not necessary to understand the details of the packaging to understand the algorithm: We store all the objects in an array; then, for each transaction, we look

through the array sequentially, from beginning to end, checking each to see whether it is the one that we seek.

To analyze the algorithm, we note immediately that the running time depends on whether or not the object sought is in the array. We can determine that the search is unsuccessful only by examining each of the N objects, but a search could end successfully at the first, second, or any one of the objects.

Therefore, the running time depends on the data. If all the searches are for the number that happens to be in the first position in the array, then the algorithm will be fast; if they are for the number that happens to be in the last position in the array, it will be slow. We discuss in Section 2.7 the distinction between being able to *guarantee* performance and being able to *predict* performance. In this case, the best guarantee that we can provide is that no more than N numbers will be examined.

To make a prediction, however, we need to make an assumption about the data. In this case, we might choose to assume that all the numbers are randomly chosen. This assumption implies, for example, that each number in the table is equally likely to be the object of a search. On reflection, we realize that it is that property of the search that is critical, because with randomly chosen numbers we would be unlikely to have a successful search at all (see Exercise 2.48). For some applications, the number of transactions that involve a successful search might be high; for other applications, it might be low. To avoid confusing the model with properties of the application, we separate the two cases (successful and unsuccessful) and analyze them independently. This example illustrates that a critical part of an effective analysis is the development of a reasonable model for the application at hand. Our analytic results will depend on the proportion of searches that are successful; indeed, it will give us information that we might need if we are to choose different algorithms for different applications based on this parameter.

Property 2.1 *Sequential search examines N numbers for each unsuccessful search and about $N/2$ numbers for each successful search on the average.*

If each number in the table is equally likely to be the object of a search, then

$$(1 + 2 + \dots + N)/N = (N + 1)/2$$

Program 2.1 Sequential search

This function checks whether the number v is among a previously stored set of numbers in $a[1]$, $a[1+1]$, ..., $a[r]$, by comparing against each number sequentially, starting at the beginning. If we reach the end without finding the number sought, then we return the value -1 . Otherwise, we return the index of the array position containing the number.

```
int search(int a[], int v, int l, int r)
{ int i;
  for (i = l; i <= r; i++)
    if (v == a[i]) return i;
  return -1;
}
```

is the average cost of a search. ■

Property 2.1 implies that the running time of Program 2.1 is proportional to N , subject to the implicit assumption that the average cost of comparing two numbers is constant. Thus, for example, we can expect that, if we double the number of objects, we double the amount of time required for a search.

We can speed up sequential search for unsuccessful search by putting the numbers in the table in order. Sorting the numbers in the table is the subject of Chapters 6 through 11. A number of the algorithms that we will consider get that task done in time proportional to $N \log N$, which is insignificant by comparison to the search costs when M is huge. In an ordered table, we can terminate the search immediately on reaching a number that is larger than the one that we seek. This change reduces the cost of sequential search to about $N/2$ numbers examined for unsuccessful search, the same as for successful search.

Property 2.2 *Sequential search in an ordered table examines N numbers for each search in the worst case and about $N/2$ numbers for each search on the average.*

We still need to specify a model for unsuccessful search. This result follows from assuming that the search is equally likely to terminate at any one of the $N + 1$ intervals defined by the N numbers in the table,

Program 2.2 Binary search

This program has the same functionality as Program 2.1, but it is much more efficient.

```
int search(int a[], int v, int l, int r)
{
    while (r >= l)
    { int m = (l+r)/2;
      if (v == a[m]) return m;
      if (v < a[m]) r = m-1; else l = m+1;
    }
    return -1;
}
```

which leads immediately to the expression

$$(1 + 2 + \dots + N + N)/N = (N + 3)/2.$$

The cost of an unsuccessful search ending before or after the N th entry in the table is the same: N . ■

Another way to state the result of Property 2.2 is to say that the running time of sequential search is proportional to MN for M transactions, on the average and in the worst case. If we double either the number of transactions or the number of objects in the table, we can expect the running time to double; if we double both, we can expect the running time to go up by a factor of 4. The result also tells us that the method is not suitable for huge tables. If it takes c microseconds to examine a single number, then, for $M = 10^9$ and $N = 10^6$, the running time for all the transactions would be at least $(c/2)10^9$ seconds, or, by Figure 2.1, about $16c$ years, which is prohibitive.

Program 2.2 is a classical solution to the search problem that is much more efficient than sequential search. It is based on the idea that, if the numbers in the table are in order, we can eliminate half of them from consideration by comparing the one that we seek with the one at the middle position in the table. If it is equal, we have a successful search. If it is less, we apply the same method to the left half of the table. If it is greater, we apply the same method to the right half of the

table. Figure 2.7 is an example of the operation of this method on a sample set of numbers.

Property 2.3 *Binary search never examines more than $\lfloor \lg N \rfloor + 1$ numbers.*

The proof of this property illustrates the use of recurrence relations in the analysis of algorithms. If we let T_N represent the number of comparisons required for binary search in the worst case, then the way in which the algorithm reduces search in a table of size N to search in a table half the size immediately implies that

$$T_N \leq T_{\lfloor N/2 \rfloor} + 1, \quad \text{for } N \geq 2 \text{ with } T_1 = 1.$$

To search in a table of size N , we examine the middle number, then search in a table of size no larger than $\lfloor N/2 \rfloor$. The actual cost could be less than this value because the comparison might cause us to terminate a successful search, or because the table to be searched might be of size $\lfloor N/2 \rfloor - 1$ (if N is even). As we did in the solution of Formula 2.2, we can prove immediately that $T_N \leq n + 1$ if $N = 2^n$ and then verify the general result by induction. ■

Property 2.3 allows us to solve a huge search problem with up to 1 million numbers with at most 20 comparisons per transaction, and that is likely to be less than the time it takes to read or write the number on many computers. The search problem is so important that several methods have been developed that are even faster than this one, as we shall see in Chapters 12 through 16.

Note that we express Property 2.1 and Property 2.2 in terms of the operations that we perform most often on the data. As we noted in the commentary following Property 2.1, we expect that each operation should take a constant amount of time, and we can conclude that the running time of binary search is proportional to $\lg N$ as compared to N for sequential search. As we double N , the running time of binary search hardly changes, but the running time of sequential search doubles. As N grows, the gap between the two methods becomes a chasm.

We can verify the analytic evidence of Properties 2.1 and 2.2 by implementing and testing the algorithms. For example, Table 2.4 shows running times for binary search and sequential search for M searches in a table of size N (including, for binary search, the cost of

1488	1488			
1578	1578			
1973	1973			
3665	3665			
4426	4426			
4548	4548			
5435	5435	5435	5435	5435
5446	5446	5446	5446	
6333	6333	6333		
6385	6385	6385		
6455	6455	6455		
6504				
6937				
6965				
7104				
7230				
8340				
8958				
9208				
9364				
9550				
9645				
9686				

Figure 2.7
Binary search
To see whether or not 5025 is in the table of numbers in the left column, we first compare it with 6504; that leads us to consider the first half of the array. Then we compare against 4548 (the middle of the first half); that leads us to the second half of the first half. We continue, always working on a subarray that would contain the number being sought, if it is in the table. Eventually, we get a subarray with just 1 element, which is not equal to 5025, so 5025 is not in the table.

Table 2.4 Empirical study of sequential and binary search

These relative timings validate our analytic results that sequential search takes time proportional to MN and binary search takes time proportional to $M \lg N$ for M searches in a table of N objects. When we increase N by a factor of 2, the time for sequential search increases by a factor of 2 as well, but the time for binary search hardly changes. Sequential search is infeasible for huge M as N increases, but binary search is fast even for huge tables.

N	$M = 1000$		$M = 10000$		$M = 100000$	
	S	B	S	B	S	B
125	1	1	13	2	130	20
250	3	0	25	2	251	22
500	5	0	49	3	492	23
1250	13	0	128	3	1276	25
2500	26	1	267	3		28
5000	53	0	533	3		30
12500	134	1	1337	3		33
25000	268	1		3		35
50000	537	0		4		39
100000	1269	1		5		47

Key:

S sequential search (Program 2.1)

B binary search (Program 2.2)

sorting the table) for various values of M and N . We will not consider the implementation of the program to run these experiments in detail here because it is similar to those that we consider in full detail in Chapters 6 and 11, and because we consider the use of library and external functions and other details of putting together programs from constituent pieces, including the sort function, in Chapter 3. For the moment, we simply stress that doing empirical testing is an integral part of evaluating the efficiency of an algorithm.

Table 2.4 validates our observation that the functional growth of the running time allows us to predict performance for huge cases

on the basis of empirical studies for small cases. The combination of mathematical analysis and empirical studies provides persuasive evidence that binary search is the preferred algorithm, by far.

This example is a prototype of our general approach to comparing algorithms. We use mathematical analysis of the frequency with which algorithms perform critical abstract operations, then use those results to deduce the functional form of the running time, which allows us to verify and extend empirical studies. As we develop algorithmic solutions to computational problems that are more and more refined, and as we develop mathematical analyses to learn their performance characteristics that are more and more refined, we call on mathematical studies from the literature, so as to keep our attention on the algorithms themselves in this book. We cannot do thorough mathematical and empirical studies of every algorithm that we encounter, but we strive to identify essential performance characteristics, knowing that, in principle, we can develop a scientific basis for making informed choices among algorithms in critical applications.

Exercises

- ▷ **2.47** Give the average number of comparisons used by Program 2.1 in the case that αN of the searches are successful, for $0 \leq \alpha \leq 1$.
- **2.48** Estimate the probability that at least one of M random 10-digit numbers matches one of a set of N given values, for $M = 10, 100$, and 1000 and $N = 10^3, 10^4, 10^5$, and 10^6 .
- 2.49** Write a driver program that generates M random integers and puts them in an array, then counts the number of N random integers that matches one of the numbers in the array, using sequential search. Run your program for $M = 10, 100$, and 1000 and $N = 10, 100$, and 1000 .
- **2.50** State and prove a property analogous to Property 2.3 for binary search.

2.7 Guarantees, Predictions, and Limitations

The running time of most algorithms depends on their input data. Typically, our goal in the analysis of algorithms is somehow to eliminate that dependence: We want to be able to say something about the performance of our programs that depends on the input data to as little an extent as possible, because we generally do not know what the input data will be each time the program is invoked. The examples

in Section 2.6 illustrate the two major approaches that we use toward this end: worst-case analysis and average-case analysis.

Studying the *worst-case* performance of algorithms is attractive because it allows us to make *guarantees* about the running time of programs. We say that the number of times certain abstract operations are executed is less than a certain function of the number of inputs, no matter what the input values are. For example, Property 2.3 is an example of such a guarantee for binary search, as is Property 1.3 for weighted quick union. If the guarantees are low, as is the case with binary search, then we are in a favorable situation, because we have eliminated cases for which our program might run slowly. Programs with good worst-case performance characteristics are a basic goal in algorithm design.

There are several difficulties with worst-case analysis, however. For a given algorithm, there might be a significant gap between the time required for it to solve a worst-case instance of the input and the time required for it to solve the data that it might encounter in practice. For example, quick union requires time proportional to N in the worst case, but only $\log N$ for typical data. More important, we cannot always prove that there is an input for which the running time of an algorithm achieves a certain bound; we can prove only that it is guaranteed to be lower than the bound. Moreover, for some problems, algorithms with good worst-case performance are significantly more complicated than are other algorithms. We often find ourselves in the position of having an algorithm with good worst-case performance that is slower than simpler algorithms for the data that occur in practice, or that is not sufficiently faster that the extra effort required to achieve good worst-case performance is justified. For many applications, other considerations—such as portability or reliability—are more important than improved worst-case performance guarantees. For example, as we saw in Chapter 1, weighted quick union with path compression provides provably better performance guarantees than weighted quick union, but the algorithms have about the same running time for typical practical data.

Studying the *average-case* performance of algorithms is attractive because it allows us to make *predictions* about the running time of programs. In the simplest situation, we can characterize precisely the inputs to the algorithm; for example, a sorting algorithm might

operate on an array of N random integers, or a geometric algorithm might process a set of N random points in the plane with coordinates between 0 and 1. Then, we calculate the average number of times that each instruction is executed, and calculate the average running time of the program by multiplying each instruction frequency by the time required for the instruction and adding them all together.

There are also several difficulties with average-case analysis, however. First, the input model may not accurately characterize the inputs encountered in practice, or there may be no natural input model at all. Few people would argue against the use of input models such as “randomly ordered file” for a sorting algorithm, or “random point set” for a geometric algorithm, and for such models it is possible to derive mathematical results that can predict accurately the performance of programs running on actual applications. But how should one characterize the input to a program that processes English-language text? Even for sorting algorithms, models other than randomly ordered inputs are of interest in certain applications. Second, the analysis might require deep mathematical reasoning. For example, the average-case analysis of union-find algorithms is difficult. Although the derivation of such results is normally beyond the scope of this book, we will illustrate their nature with a number of classical examples, and we will cite relevant results when appropriate (fortunately, many of our best algorithms have been analyzed in the research literature). Third, knowing the average value of the running time might not be sufficient: we may need to know the standard deviation or other facts about the distribution of the running time, which may be even more difficult to derive. In particular, we are often interested in knowing the chance that the algorithm could be dramatically slower than expected.

In many cases, we can answer the first objection listed in the previous paragraph by turning randomness to our advantage. For example, if we randomly scramble an array before attempting to sort it, then the assumption that the elements in the array are in random order is accurate. For such algorithms, which are called *randomized algorithms*, the average-case analysis leads to predictions of the expected running time in a strict probabilistic sense. Moreover, we are often able to prove that the probability that such an algorithm will be slow is negligibly small. Examples of such algorithms include quicksort

(see Chapter 9), randomized BSTs (see Chapter 13), and hashing (see Chapter 14).

The field of *computational complexity* is the branch of analysis of algorithms that helps us to understand the fundamental *limitations* that we can expect to encounter when designing algorithms. The overall goal is to determine the worst-case running time of the *best* algorithm to solve a given problem, to within a constant factor. This function is called the *complexity* of the problem.

Worst-case analysis using the O -notation frees the analyst from considering the details of particular machine characteristics. The statement that the running time of an algorithm is $O(f(N))$ is independent of the input and is a useful way to categorize algorithms in a way that is independent of both inputs and implementation details, separating the analysis of an algorithm from any particular implementation. We ignore constant factors in the analysis; in most cases, if we want to know whether the running time of an algorithm is proportional to N or proportional to $\log N$, it does not matter whether the algorithm is to be run on a nanocomputer or on a supercomputer, and it does not matter whether the inner loop has been implemented carefully with only a few instructions or badly implemented with many instructions.

When we can prove that the worst-case running time of an algorithm to solve a certain problem is $O(f(N))$, we say that $f(N)$ is an *upper bound* on the complexity of the problem. In other words, the running time of the best algorithm to solve a problem is no higher than the running time of any particular algorithm to solve the problem.

We constantly strive to improve our algorithms, but we eventually reach a point where no change seems to improve the running time. For every given problem, we are interested in knowing when to stop trying to find improved algorithms, so we seek *lower bounds* on the complexity. For many problems, we can prove that *any* algorithm to solve the problem must use a certain number of fundamental operations. Proving lower bounds is a difficult matter of carefully constructing a machine model and then developing intricate theoretical constructions of inputs that are difficult for any algorithm to solve. We rarely touch on the subject of proving lower bounds, but they represent computational barriers that guide us in the design of algorithms, so we maintain awareness of them when they are relevant.

When complexity studies show that the upper bound of an algorithm matches the lower bound, then we have some confidence that it is fruitless to try to design an algorithm that is fundamentally faster than the best known, and we can start to concentrate on the implementation. For example, binary search is optimal, in the sense that no algorithm that uses comparisons exclusively can use fewer comparisons in the worst case than binary search.

We also have matching upper and lower bounds for pointer-based union-find algorithms. Tarjan showed in 1975 that weighted quick union with path compression requires following less than $O(\lg^* V)$ pointers in the worst case, and that any pointer-based algorithm must follow more than a constant number of pointers in the worst case for some input. In other words, there is no point looking for some new improvement that will guarantee to solve the problem with a linear number of $i = a[i]$ operations. In practical terms, this difference is hardly significant, because $\lg^* V$ is so small; still, finding a simple linear algorithm for this problem was a research goal for many years, and Tarjan's lower bound has allowed researchers to move on to other problems. Moreover, the story shows that there is no avoiding functions like the rather complicated $\log^*$ function, because such functions are intrinsic to this problem.

Many of the algorithms in this book have been subjected to detailed mathematical analyses and performance studies far too complex to be discussed here. Indeed, it is on the basis of such studies that we are able to recommend many of the algorithms that we discuss.

Not all algorithms are worthy of such intense scrutiny; indeed, during the design process, it is preferable to work with approximate performance indicators to guide the design process without extraneous detail. As the design becomes more refined, so must the analysis, and more sophisticated mathematical tools need to be applied. Often, the design process leads to detailed complexity studies that lead to theoretical algorithms that are rather far from any particular application. It is a common mistake to assume that rough analyses from complexity studies will translate immediately into efficient practical algorithms; such assumptions can lead to unpleasant surprises. On the other hand, computational complexity is a powerful tool that tells us when we have reached performance limits in our design work and that

can suggest departures in design in pursuit of closing the gap between upper and lower bounds.

In this book, we take the view that algorithm design, careful implementation, mathematical analysis, theoretical studies, and empirical analysis all contribute in important ways to the development of elegant and efficient programs. We want to gain information about the properties of our programs using any tools at our disposal, then to modify or develop new programs on the basis of that information. We will not be able to do exhaustive testing and analysis of every algorithm that we run in every programming environment on every machine, but we can use careful implementations of algorithms that we know to be efficient, then refine and compare them when peak performance is necessary. Throughout the book, when appropriate, we shall consider the most important methods in sufficient detail to appreciate why they perform well.

Exercise

- 2.51 You are given the information that the time complexity of one problem is $N \log N$ and that the time complexity of another problem is N^3 . What does this statement imply about the relative performance of specific algorithms that solve the problems?

References for Part One

There are a large number of introductory textbooks on programming. Still, the best source for specific facts about C and examples of C programs, in the same spirit as those found in this book, is Kernighan and Ritchie's book on the language.

The many variants on algorithms for the union-find problem of Chapter 1 are ably categorized and compared by van Leeuwen and Tarjan.

Bentley's books describe, again in the same spirit as much of the material here, a number of detailed case studies on evaluating various approaches to developing algorithms and implementations for solving numerous interesting problems.

The classic reference on the analysis of algorithms based on asymptotic worst-case performance measures is Aho, Hopcroft, and Ullman's book. Knuth's books cover average-case analysis more fully and are the authoritative source on specific properties of numerous algorithms. The books by Gonnet and Baeza-Yates and by Cormen, Leiserson, and Rivest are more recent works; both include extensive references to the research literature.

The book by Graham, Knuth and Patashnik covers the type of mathematics that commonly arises in the analysis of algorithms, and such material is also sprinkled liberally throughout Knuth's books. The book by Sedgewick and Flajolet is a thorough introduction to the subject.

- A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Algorithms*, Addison-Wesley, Reading, MA, 1975.
- J. L. Bentley, *Programming Pearls*, Addison-Wesley, Reading, MA, 1985; *More Programming Pearls*, Addison-Wesley, Reading, MA, 1988.
- R. Baeza-Yates and G. H. Gonnet, *Handbook of Algorithms and Data Structures*, second edition, Addison-Wesley, Reading, MA, 1984.
- T. H. Cormen, C. E. Leiserson, and R. L. Rivest, *Introduction to Algorithms*, MIT Press/McGraw-Hill, Cambridge, MA, 1990.
- R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics*, Addison-Wesley, Reading, MA, 1988.

- B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, second edition, Prentice-Hall, Englewood Cliffs, NJ, 1988.
- D. E. Knuth, *The Art of Computer Programming. Volume 1: Fundamental Algorithms*, second edition, Addison-Wesley, Reading, MA, 1973; *Volume 2: Seminumerical Algorithms*, second edition, Addison-Wesley, Reading, MA, 1981; *Volume 3: Sorting and Searching*, second printing, Addison-Wesley, Reading, MA, 1975.
- R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, Addison-Wesley, Reading, MA, 1996.
- J. van Leeuwen and R. E. Tarjan, "Worst-case analysis of set-union algorithms," *Journal of the ACM*, 1984.

P A R T
T W O

Data Structures

CHAPTER THREE

Elementary Data Structures

ORGANIZING THE DATA for processing is an essential step in the development of a computer program. For many applications, the choice of the proper data structure is the only major decision involved in the implementation: once the choice has been made, the necessary algorithms are simple. For the same data, some data structures require more or less space than others; for the same operations on the data, some data structures lead to more or less efficient algorithms than others. The choices of algorithm and of data structure are closely intertwined, and we continually seek ways to save time or space by making the choice properly.

A data structure is not a passive object: We also must consider the operations to be performed on it (and the algorithms used for these operations). This concept is formalized in the notion of a *data type*. In this chapter, our primary interest is in concrete implementations of the fundamental approaches that we use to structure data. We consider basic methods of organization and methods for manipulating data, work through a number of specific examples that illustrate the benefits of each, and discuss related issues such as storage management. In Chapter 4, we discuss *abstract data types*, where we separate the definitions of data types from implementations.

We discuss properties of arrays, linked lists, and strings. These classical data structures have widespread applicability: with trees (see Chapter 5), they form the basis for virtually all the algorithms considered in this book. We consider various primitive operations for manipulating these data structures, to develop a basic set of tools that we can use to develop sophisticated algorithms for difficult problems.

The study of storing data as variable-sized objects and in linked data structures requires an understanding of how the system manages the storage that it allocates to programs for their data. We do not cover this subject exhaustively because many of the important considerations are system and machine dependent. However, we do discuss approaches to storage management and several basic underlying mechanisms. Also, we discuss the specific (stylized) manners in which we will be using C storage-allocation mechanisms in our programs.

At the end of the chapter, we consider several examples of *compound structures*, such as arrays of linked lists and arrays of arrays. The notion of building abstract mechanisms of increasing complexity from lower-level ones is a recurring theme throughout this book. We consider a number of examples that serve as the basis for more advanced algorithms later in the book.

The data structures that we consider in this chapter are important building blocks that we can use in a natural manner in C and many other programming languages. In Chapter 5, we consider another important data structure, the *tree*. Arrays, strings, linked lists, and trees are the basic elements underlying most of the algorithms that we consider in this book. In Chapter 4, we discuss the use of the concrete representations developed here in building basic abstract data types that can meet the needs of a variety of applications. In the rest of the book, we develop numerous variations of the basic tools discussed here, trees, and abstract data types, to create algorithms that can solve more difficult problems and that can serve us well as the basis for higher-level abstract data types in diverse applications.

3.1 Building Blocks

In this section, we review the primary low-level constructs that we use to store and process information in C. All the data that we process on a computer ultimately decompose into individual bits, but writing programs that exclusively process bits would be tiresome indeed. *Types* allow us to specify how we will use particular sets of bits and *functions* allow us to specify the operations that we will perform on the data. We use C *structures* to group together heterogeneous pieces of information, and we use *pointers* to refer to information indirectly. In this section, we consider these basic C mechanisms, in the context of presenting a

general approach to organizing our programs. Our primary goal is to lay the groundwork for the development, in the rest of the chapter and in Chapters 4 and 5, of the higher-level constructs that will serve as the basis for most of the algorithms that we consider in this book.

We write programs that process information derived from mathematical or natural-language descriptions of the world in which we live; accordingly, computing environments need to provide built-in support for the basic building blocks of such descriptions—numbers and characters. In C, our programs are all built from just a few basic types of data:

- Integers (`ints`).
- Floating-point numbers (`floats`).
- Characters (`chars`).

It is customary to refer to these basic types by their C names—`int`, `float`, and `char`—although we often use the generic terminology *integer*, *floating-point number*, and *character*, as well. Characters are most often used in higher-level abstractions—for example to make words and sentences—so we defer consideration of character data to Section 3.6 and look at numbers here.

We use a fixed number of bits to represent numbers, so `ints` are by necessity integers that fall within a specific range that depends on the number of bits that we use to represent them. Floating-point numbers approximate real numbers, and the number of bits that we use to represent them affects the precision with which we can approximate a real number. In C, we trade space for accuracy by choosing from among the types `int`, `long int`, or `short int` for integers and from among `float` or `double` for floating-point numbers. On most systems, these types correspond to underlying hardware representations. The number of bits used for the representation, and therefore the range of values (in the case of `ints`) or precision (in the case of `floats`), is machine-dependent (see Exercise 3.1), although C provides certain guarantees. In this book, for clarity, we normally use `int` and `float`, except in cases where we want to emphasize that we are working with problems where big numbers are needed.

In modern programming, we think of the type of the data more in terms of the needs of the program than the capabilities of the machine, primarily, in order to make programs portable. Thus, for example, we think of a `short int` as an object that can take on values between

–32,768 and 32,767, instead of as a 16-bit object. Moreover, our concept of an integer includes the operations that we perform on them: addition, multiplication, and so forth.

Definition 3.1 *A data type is a set of values and a collection of operations on those values.*

Operations are associated with types, not the other way around. When we perform an operation, we need to ensure that its operands and result are of the correct type. Neglecting this responsibility is a common programming error. In some situations, C performs implicit type conversions; in other situations, we use *casts*, or explicit type conversions. For example, if *x* and *N* are integers, the expression

`((float) x) / N`

includes both types of conversion: the `(float)` is a cast that converts the value of *x* to floating point; then an implicit conversion is performed for *N* to make both arguments of the divide operator floating point, according to C's rules for implicit type conversion.

Many of the operations associated with standard data types (for example, the arithmetic operations) are built into the C language. Other operations are found in the form of functions that are defined in standard function libraries; still others take form in the C functions that we define in our programs (see Program 3.1). That is, the concept of a data type is relevant not just to integer, floating point, and character built-in types. We often define our own data types, as an effective way of organizing our software. When we define a simple function in C, we are effectively creating a new data type, with the operation implemented by that function added to the operations defined for the types of data represented by its arguments. Indeed, in a sense, *each* C program is a data type—a list of sets of values (built-in or other types) and associated operations (functions). This point of view is perhaps too broad to be useful, but we shall see that narrowing our focus to understand our programs in terms of data types is valuable.

One goal that we have when writing programs is to organize them such that they apply to as broad a variety of situations as possible. The reason for adopting such a goal is that it might put us in the position of being able to reuse an old program to solve a new problem, perhaps completely unrelated to the problem that the program was originally intended to solve. First, by taking care to understand and to specify

Program 3.1 Function definition

The mechanism that we use in C to implement new operations on data is the *function definition*, illustrated here.

All functions have a list of *arguments* and possibly a *return value*. The function `lg` here has one argument and a return value, each of type `int`. The function `main` has neither arguments nor return value.

We *declare* the function by giving its name and the types of its return values. The first line of code here references a system file that contains declarations of system functions such as `printf`. The second line of code is a declaration for `lg`. The declaration is optional if the function is defined (see next paragraph) before it is used, as is the case with `main`. The declaration provides the information necessary for other functions to *call* or *invoke* the function, using arguments of the proper type. The calling function can use the function in an expression, in the same way as it uses variables of the return-value type.

We *define* functions with C code. All C programs include a definition of the function `main`, and this code also defines `lg`. In a function definition, we give names to the arguments (which we refer to as *parameters*) and express the computation in terms of those names, as if they were local variables. When the function is invoked, these variables are initialized with the values of the arguments and the function code is executed. The `return` statement is the instruction to end execution of the function and provide the return value to the calling function. In principle, the calling function is not to be otherwise affected, though we shall see many exceptions to this principle.

The separation of definition and declaration provides flexibility in organizing programs. For example, both could be in separate files (*see text*). Or, in a simple program like this one, we could put the definition of `lg` before the definition of `main` and omit its declaration.

```
#include <stdio.h>
int lg(int);
main()
{ int i, N;
  for (i = 1, N = 10; i <= 6; i++, N *= 10)
    printf("%7d %2d %9d\n", N, lg(N), N*lg(N));
}
int lg(int N)
{ int i;
  for (i = 0; N > 0; i++, N /= 2) ;
  return i;
}
```


precisely which operations a program uses, we can easily extend it to any type of data for which we can support those operations. Second, by taking care to understand and to specify precisely what a program does, we can add the abstract operation that it performs to the operations at our disposal in solving new problems.

Program 3.2 implements a simple computation on numbers using a simple data type defined with a typedef operation and a function (which itself is implemented with a library function). The main function refers to the data type, not the built-in type of the number. By not specifying the type of the numbers that the program processes, we extend its potential utility. For example, this practice is likely to extend the useful lifetime of a program. When some new circumstance (a new application, or perhaps a new compiler or computer), presents us with a new type of number with which we would like to work, we can update our program just by changing the data type.

This example does not represent a fully general solution to the problem of developing a type-independent program for computing averages and standard deviations—nor is it intended to do so. For example, the program depends on converting a number of type `Number` to a `float` to be included in the running average and variance, so we might add that conversion as an operation to the data type, rather than depend on the `(float)` cast, which only works for built-in types of numbers.

If we were to try to do operations other than arithmetic operations, we would soon find the need to add more operations to the data type. For example, we might want to print the numbers, which would require that we implement, say, a `printNum` function. Such a function would be less convenient than using the built-in format conversions in `printf`. Whenever we strive to develop a data type based on identifying the operations of importance in a program, we need to strike a balance between the level of generality that we choose and the ease of implementation and use that results.

It is worthwhile to consider in detail how we might change the data type to make Program 3.2 work with other types of numbers, say `floats`, rather than with `ints`. There are a number of different mechanisms available in C that we could use to take advantage of the fact that we have localized references to the type of the data. For such

Program 3.2 Types of numbers

This program computes the average μ and standard deviation σ of a sequence $x_1, x_2, \dots, x_N$ of integers generated by the library procedure `rand`, following the mathematical definitions

$$\mu = \frac{1}{N} \sum_{1 \leq i \leq N} x_i \quad \text{and} \quad \sigma^2 = \frac{1}{N} \sum_{1 \leq i \leq N} (x_i - \mu)^2 = \frac{1}{N} \sum_{1 \leq i \leq N} x_i^2 - \mu^2.$$

Note that a direct implementation from the definition of σ^2 requires one pass to compute the average and another to compute the sums of the squares of the differences between the members of the sequence and the average, but rearranging the formula makes it possible for us to compute σ^2 in one pass through the data.

We use the `typedef` declaration to localize reference to the fact that the type of the data is `int`. For example, we could keep the `typedef` and the function `randNum` in a separate file (referenced by an `include` directive), and then we could use this program to test random numbers of a different type by changing that file (*see text*).

Whatever the type of the data, the program uses `ints` for indices and `floats` to compute the average and standard deviation, and will be effective only if conversion functions from the data to `float` perform in a reasonable manner.

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
typedef int Number;
Number randNum()
{ return rand(); }
main(int argc, char *argv[])
{ int i, N = atoi(argv[1]);
  float m1 = 0.0, m2 = 0.0;
  Number x;
  for (i = 0; i < N; i++)
  {
    x = randNum();
    m1 += ((float) x)/N;
    m2 += ((float) x*x)/N;
  }
  printf("      Average: %f\n", m1);
  printf("Std. deviation: %f\n", sqrt(m2-m1*m1));
}
```

a small program, the simplest is to make a copy of the file, then to change the typedef to

```
typedef float Number
```

and the function randNum to

```
return 1.0*rand()/RAND_MAX;
```

(which will return random floating-point numbers between 0 and 1). Even for such a small program, this approach is inconvenient because it leaves us with two copies of the main program, and we will have to make sure that any later changes in that program are reflected in both copies. In C, an alternative approach is to put the typedef and randNum into a separate *header file*—called, say, Num.h—replacing them with the directive

```
#include "Num.h"
```

in the code in Program 3.2. Then, we can make a second header file with different typedef and randNum, and, by renaming one of these files or the other Num.h, use the main program in Program 3.2 with either, without modifying *it* at all.

A third alternative, which is recommended software engineering practice, is to split the program into *three* files:

- An *interface*, which defines the data structure and declares the functions to be used to manipulate the data structure
- An *implementation* of the functions declared in the interface
- A *client* program that uses the functions declared in the interface to work at a higher level of abstraction

With this arrangement, we can use the main program in Program 3.2 with integers or floats, or extend it to work with other data types, just by compiling it together with the specific code for the data type of interest. Next, we shall consider the precise change that we need to convert Program 3.2 into a more flexible implementation, using this approach.

We think of the interface as a definition of the data type. It is a contract between the client program and the implementation program. The client agrees to access the data only through the functions defined in the interface, and the implementation agrees to deliver the promised functions.

For the example in Program 3.2, the *interface* would consist of the declarations

```
typedef int Number;  
Number randNum();
```

The first line specifies the type of the data to be processed, and the second specifies an operation associated with the type. This code might be kept, for example, in a file named `Num.h`.

The *implementation* of the interface in `Num.h` is an implementation of the `randNum` function, which might consist of the code

```
#include <stdlib.h>  
#include "Num.h"  
Number randNum()  
{ return rand(); }
```

The first line refers to the system-supplied interface that describes the `rand()` function; the second line refers to the interface that we are implementing (we include it as a check that the function we are implementing is the same type as the one that we declared), and the final two lines give the code for the function. This code might be kept, for example, in a file named `int.c`. The actual code for the `rand` function is kept in the standard C run-time library.

A *client* program corresponding to Program 3.2 would begin with the include directives for interfaces that declare the functions that it uses, as follows:

```
#include <stdio.h>  
#include <math.h>  
#include "Num.h"
```

The function `main` from Program 3.2 then can follow these three lines. This code might be kept, for example, in a file named `avg.c`.

Compiled together, the programs `avg.c` and `int.c` described in the previous paragraphs have the same functionality as Program 3.2, but they represent a more flexible implementation both because the code associated with the data type is encapsulated and can be used by other client programs and because `avg.c` can be used with other data types without being changed.

There are many other ways to support data types besides the client–interface–implementation scenario just described, but we will not dwell on distinctions among various alternatives because such

distinctions are best drawn in a systems-programming context, rather than in an algorithm-design context (*see reference section*). However, we do often make use of this basic design paradigm because it provides us with a natural way to substitute improved implementations for old ones, and therefore to compare different algorithms for the same applications problem. Chapter 4 is devoted to this topic.

We often want to build data structures that allow us to handle collections of data. The data structures may be huge, or they may be used extensively, so we are interested in identifying the important operations that we will perform on the data and in knowing how to implement those operations efficiently. Doing these tasks is taking the first steps in the process of incrementally building lower-level abstractions into higher-level ones; that process allows us to conveniently develop ever more powerful programs. The simplest mechanisms for grouping data in an organized way in C are *arrays*, which we consider in Section 3.2, and *structures*, which we consider next.

Structures are aggregate types that we use to define collections of data such that we can manipulate an entire collection as a unit, but can still refer to individual components of a given datum by name. Structures are not at the same level as built-in types such as `int` or `float` in C, because the only operations that are defined for them (beyond referring to their components) are copy and assignment. Thus, we can use a structure to define a new type of data, and can use it to name variables, and can pass those variables as arguments to functions, but we have specifically to define as functions any operations that we want to perform.

For example, when processing geometric data we might want to work with the abstract notion of points in the plane. Accordingly, we can write

```
struct point { float x; float y; };
```

to indicate that we will use type `point` to refer to pairs of floating-point numbers. For example, the statement

```
struct point a, b;
```

declares two variables of this type. We can refer to individual members of a structure by name. For example, the statements

```
a.x = 1.0; a.y = 1.0; b.x = 4.0; b.y = 5.0;
```

set `a` to represent the point (1, 1) and `b` to represent the point (4, 5).

Program 3.3 Point data type interface

This interface defines a data type consisting of the set of values “pairs of floating-point numbers” and the operation consists of a function that computes the distance between two points.

```
typedef struct { float x; float y; } point;  
float distance(point, point);
```

We can also pass structures as arguments to functions. For example, the code

```
float distance(struct point a, struct point b)  
{ float dx = a.x - b.x, dy = a.y - b.y;  
  return sqrt(dx*dx + dy*dy);  
}
```

defines a function that computes the distance between two points in the plane. This example illustrates the natural way in which structures allow us to aggregate our data in typical applications.

Program 3.3 is an interface that embodies the definition of a data type for points in the plane, uses a structure to represent the points, and includes an operation to compute the distance between two points. Program 3.4 is a function that implements the operation. We use interface-implementation arrangements like this to define data types whenever possible, because they encapsulate the definition (in the interface) and the implementation in a clear and direct manner. We make use of the data type in a client program by including the interface and by compiling the implementation with the client program (or by using appropriate separate-compilation facilities). Program 3.4 uses a typedef to define the point data type so that client programs can declare points as `point` instead of `struct point`, and do not have to make any assumptions about how the data types are represented. In Chapter 4, we shall see how to carry this separation between client and implementation one step further.

We cannot use Program 3.2 to process items of type `point` because arithmetic and type conversion operations are not defined for points. Modern languages such as C++ and Java have basic constructs that make it possible to use previously defined high-level abstract operations, even for newly defined types. With a sufficiently general

Program 3.4 Point data type implementation

This implementation provides the definition for the distance function for points that is declared in Program 3.3. It makes use of a library function to compute the square root.

```
#include <math.h>
#include "Point.h"
float distance(point a, point b)
{ float dx = a.x - b.x, dy = a.y - b.y;
  return sqrt(dx*dx + dy*dy);
}
```

interface, we could make these arrangements, even in C. In this book, however, although we strive to develop interfaces of general utility, we resist obscuring our algorithms or sacrificing good performance for that reason. Our primary goal is to make clear the effectiveness of the algorithmic ideas that we will be considering. Although we often stop short of a fully general solution, we do pay careful attention to the process of precisely defining the abstract operations that we want to perform, as well as the data structures and algorithms that will support those operations, because doing so is at the heart of developing efficient and effective programs. We will return to this issue, in detail, in Chapter 4.

The point structure example just given is a simple one that comprises two items of the same type. In general, structures can mix different types of data. We shall be working extensively with such structures throughout the rest of this chapter.

Beyond giving us the specific basic types `int`, `float`, and `char`, and the ability to build them into compound types with `struct`, C provides us with the ability to manipulate our data indirectly. A *pointer* is a reference to an object in memory (usually implemented as a machine address). We declare a variable `a` to be a pointer to (for example) an integer by writing `int *a`, and we can refer to the integer itself as `*a`. We can declare pointers to any type of data. The unary operator `&` gives the machine address of an object, and is useful for initializing pointers. For example, `*&a` is the same as `a`. We restrict ourselves to using `&` for this purpose, as we prefer to work at a somewhat higher level of abstraction than machine addresses when possible.

objects in a fixed sequential fashion that is more suitable for access than for manipulation; or a *list*, where we organize objects in a logical sequential fashion that is more suitable for manipulation than for access.

Exercises

- ▷ 3.1 Find the largest and smallest numbers that you can represent with types `int`, `long int`, `short int`, `float`, and `double` in your programming environment.
- 3.2 Test the random-number generator on your system by generating N random integers between 0 and $r - 1$ with `rand() % r` and computing the average and standard deviation for $r = 10, 100$, and 1000 and $N = 10^3, 10^4, 10^5$, and 10^6 .
- 3.3 Test the random-number generator on your system by generating N random numbers of type `double` between 0 and 1, transforming them to integers between 0 and $r - 1$ by multiplying by r and truncating the result, and computing the average and standard deviation for $r = 10, 100$, and 1000 and $N = 10^3, 10^4, 10^5$, and 10^6 .
- 3.4 Do Exercises 3.2 and 3.3 for $r = 2, 4$, and 16 .
- 3.5 Implement the necessary functions to allow Program 3.2 to be used for random *bits* (numbers that can take only the values 0 or 1).
- 3.6 Define a `struct` suitable for representing a playing card.
- 3.7 Write a client program that uses the data type in Programs 3.3 and 3.4 for the following task: Read a sequence of points (pairs of floating-point numbers) from standard input, and find the one that is closest to the first.
- 3.8 Add a function to the point data type (Programs 3.3 and 3.4) that determines whether or not three points are collinear, to within a numerical tolerance of 10^{-4} . Assume that the points are all in the unit square.
- 3.9 Define a data type for points in the plane that is based on using polar coordinates instead of Cartesian coordinates.
- 3.10 Define a data type for *triangles* in the unit square, including a function that computes the area of a triangle. Then write a client program that generates random triples of pairs of `floats` between 0 and 1 and computes empirically the average area of the triangles generated.

3.2 Arrays

Perhaps the most fundamental data structure is the *array*, which is defined as a primitive in C and in most other programming languages.

We have already seen the use of an array as the basis for the development of an efficient algorithm, in the examples in Chapter 1; we shall see many more examples in this section.

An array is a fixed collection of same-type data that are stored contiguously and that are accessible by an index. We refer to the i th element of an array a as $a[i]$. It is the responsibility of the programmer to store something meaningful in an array position $a[i]$ before referring to $a[i]$. In C, it is also the responsibility of the programmer to use indices that are nonnegative and smaller than the array size. Neglecting these responsibilities are two of the more common programming mistakes.

Arrays are fundamental data structures in that they have a direct correspondence with memory systems on virtually all computers. To retrieve the contents of a word from memory in machine language, we provide an address. Thus, we could think of the entire computer memory as an array, with the memory addresses corresponding to array indices. Most computer-language processors translate programs that involve arrays into efficient machine-language programs that access memory directly, and we are safe in assuming that an array access such as $a[i]$ translates to just a few machine instructions.

A simple example of the use of an array is given by Program 3.5, which prints out all prime numbers less than 10000. The method used, which dates back to the third century B.C., is called the *sieve of Eratosthenes* (see Figure 3.1). It is typical of algorithms that exploit the fact that we can access efficiently any item of an array, given that item's index. The implementation has four loops, three of which access the items of the array sequentially, from beginning to end; the fourth skips through the array, i items at a time. In some cases, sequential processing is essential; in other cases, sequential ordering is used because it is as good as any other. For example, we could change the first loop in Program 3.5 to

```
for (a[1] = 0, i = N-1; i > 1; i--) a[i] = 1;
```

without any effect on the computation. We could also reverse the order of the inner loop in a similar manner, or we could change the final loop to print out the primes in decreasing order, but we could not change the order of the outer loop in the main computation, because it depends on all the integers less than i being processed before $a[i]$ is tested for being prime.

i	2	3	5	a[i]
2	1			1
3	1			1
4	1	0		
5	1			1
6	1	0		
7	1			1
8	1	0		
9	1	0		
10	1	0		
11	1			1
12	1	0	0	
13	1			1
14	1	0		
15	1	0		
16	1	0		
17	1			1
18	1	0	0	
19	1			1
20	1	0		
21	1	0		
22	1	0		
23	1			1
24	1	0	0	
25	1		0	
26	1	0		
27	1	0		
28	1	0		
29	1			1
30	1	0	0	0
31	1			1

Figure 3.1
Sieve of Eratosthenes

To compute the prime numbers less than 32, we initialize all the array entries to 1 (second column), to indicate that no numbers are known to be nonprime ($a[0]$ and $a[1]$ are not used and are not shown). Then, we set array entries whose indices are multiples of 2, 3, and 5 to 0, since we know these multiples to be nonprime. Indices corresponding to array entries that remain 1 are prime (rightmost column).

Program 3.5 Sieve of Eratosthenes

The goal of this program is to set $a[i]$ to 1 if i is prime, and to 0 if i is not prime. First, it sets to 1 all array elements, to indicate that no numbers are known to be nonprime. Then it sets to 0 array elements corresponding to indices that are known to be nonprime (multiples of known primes). If $a[i]$ is still 1 after all multiples of smaller primes have been set to 0, then we know it to be prime.

Because the program uses an array consisting of the simplest type of elements, 0–1 values, it would be more space efficient if we explicitly used an array of bits, rather than one of integers. Also, some programming environments might require the array to be global if N is huge, or we could allocate it dynamically (see Program 3.6).

```
#define N 10000
main()
{ int i, j, a[N];
  for (i = 2; i < N; i++) a[i] = 1;
  for (i = 2; i < N; i++)
    if (a[i])
      for (j = i; i*j < N; j++) a[i*j] = 0;
  for (i = 2; i < N; i++)
    if (a[i]) printf("%4d ", i);
  printf("\n");
}
```

We will not analyze the running time of Program 3.5 in detail because that would take us astray into number theory, but it is clear that the running time is proportional to

$$N + N/2 + N/3 + N/5 + N/7 + N/11 + \dots$$

which is less than $N + N/2 + N/3 + N/4 + \dots = NH_N \sim N \ln N$.

One of the distinctive features of C is that an array name generates a pointer to the first element of the array (the one with index 0). Moreover, simple *pointer arithmetic* is allowed: if p is a pointer to an object of a certain type, then we can write code that assumes that objects of that type are arranged sequentially, and can use $*p$ to refer to the first object, $*(p+1)$ to refer to the second object, $*(p+2)$ to refer to the third object, and so forth. In other words,

$*(a+i)$ and $a[i]$ are equivalent in C.

Program 3.6 Dynamic memory allocation for an array

To change the value of the maximum prime computed in Program 3.5, we need to recompile the program. Instead, we can take the maximum desired number from the command line, and use it to allocate space for the array at execution time, using the library function `malloc` from `stdlib.c`. For example, if we compile this program and use 1000000 as a command-line argument, then we get all the primes less than 1 million (as long as our computer is big and fast enough to make the computation feasible); we can also debug with 100 (without using much time or space). We will use this idiom frequently, though, for brevity, we will omit the insufficient-memory test.

```
#include <stdlib.h>
main(int argc, char *argv[])
{ long int i, j, N = atoi(argv[1]);
  int *a = malloc(N*sizeof(int));
  if (a == NULL)
    { printf("Insufficient memory.\n"); return; }
  ...
```

This equivalence provides an alternate mechanism for accessing objects in arrays that is sometimes more convenient than indexing. This mechanism is most often used for arrays of characters (strings); we discuss it again in Section 3.6.

Like structures, pointers to arrays are significant because they allow us to manipulate the arrays efficiently as higher-level objects. In particular, we can pass a pointer to an array as an argument to a function, thus enabling that function to access objects in the array without having to make a copy of the whole array. This capability is indispensable when we write programs to manipulate huge arrays. For example, the search functions that we examined in Section 2.6 use this feature. We shall see other examples in Section 3.7.

The implementation in Program 3.5 assumes that the size of the array must be known beforehand: to run the program for a different value of `N`, we must change the constant `N` and recompile the program before executing it. Program 3.6 shows an alternate approach, where a user of the program can type in the value of `N`, and it will respond with the primes less than `N`. It uses two basic C mechanisms, both of which involve passing arrays as arguments to functions. The first is the

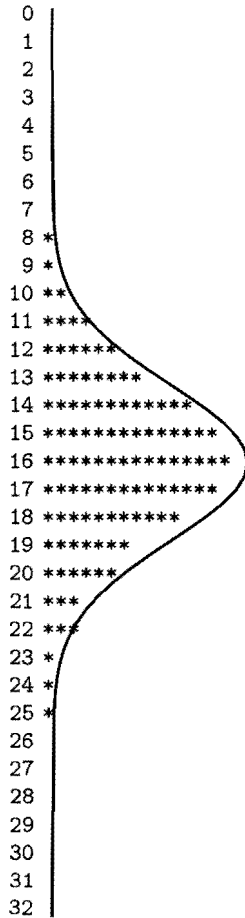


Figure 3.2
Coin-flipping simulation

This table shows the result of running Program 3.7 with $N = 32$ and $M = 1000$, simulating 1000 experiments of flipping a coin 32 times. The number of heads that we should see is approximated by the normal distribution function, which is drawn over the data.

mechanism by which command-line arguments are passed to the main program, in an array `argv` of size `argc`. The array `argv` is a compound array made up of objects that are arrays (strings) themselves, so we shall defer discussing it in further detail until Section 3.7, and shall take on faith for the moment that the variable `N` gets the number that the user types when executing the program.

The second basic mechanism that we use in Program 3.6 is `malloc`, a function that *allocates* the amount of memory that we need for our array at execution time, and returns, for our exclusive use, a pointer to the array. In some programming languages, it is difficult or impossible to allocate arrays dynamically; in some other programming languages, memory allocation is an automatic mechanism. Dynamic allocation is an essential tool in programs that manipulate multiple arrays, some of which might have to be huge. In this case, without memory allocation, we would have to predeclare an array as large as any value that the user is allowed to type. In a large program where we might use many arrays, it is not feasible to do so for each array. We will generally use code like Program 3.6 in this book because of the flexibility that it provides, although in specific applications when the array size is known, simpler versions like Program 3.5 are perfectly suitable. If the array size is fixed and huge, the array may need to be global in some systems. We discuss several of the mechanisms behind memory allocation in Section 3.5, and we look at a way to use `malloc` to support an abstract dynamic growth facility for arrays in Section 14.5. As we shall see, however, such mechanisms have associated costs, so we generally regard arrays as having the characteristic property that, once allocated, their sizes are fixed, and cannot be changed.

Not only do arrays closely reflect the low-level mechanisms for accessing data in memory on most computers, but also they find widespread use because they correspond directly to natural methods of organizing data for applications. For example, arrays also correspond directly to *vectors*, the mathematical term for indexed lists of objects.

Program 3.7 is an example of a simulation program that uses an array. It simulates a sequence of *Bernoulli trials*, a familiar abstract concept from probability theory. If we flip a coin N times, the probability that we see k heads is

$$\binom{N}{k} \frac{1}{2^N} \approx \frac{e^{-(k-N/2)^2/N}}{\sqrt{\pi N/2}}.$$

Program 3.7 Coin-flipping simulation

If we flip a coin N times, we expect to get $N/2$ heads, but could get anywhere from 0 to N heads. This program runs the experiment M times, taking both N and M from the command line. It uses an array f to keep track of the frequency of occurrence of the outcome “ i heads” for $0 \leq i \leq N$, then prints out a histogram of the result of the experiments, with one asterisk for each 10 occurrences.

The operation on which this program is based—indexing an array with a computed value—is critical to the efficiency of many computational procedures.

```
#include <stdlib.h>
int heads()
{ return rand() < RAND_MAX/2; }
main(int argc, char *argv[])
{ int i, j, cnt;
  int N = atoi(argv[1]), M = atoi(argv[2]);
  int *f = malloc((N+1)*sizeof(int));
  for (j = 0; j <= N; j++) f[j] = 0;
  for (i = 0; i < M; i++, f[cnt]++)
    for (cnt = 0, j = 0; j <= N; j++)
      if (heads()) cnt++;
  for (j = 0; j <= N; j++)
  {
    printf("%2d ", j);
    for (i = 0; i < f[j]; i+=10) printf("*");
    printf("\n");
  }
}
```

The approximation is known as the *normal approximation*: the familiar bell-shaped curve. Figure 3.2 illustrates the output of Program 3.7 for 1000 trials of the experiment of flipping a coin 32 times. Many more details on the Bernoulli distribution and the normal approximation can be found in any text on probability, and we shall encounter these distributions again in Chapter 13. In the present context, our interest in the computation is that we use the numbers as indices into an array to count their frequency of occurrence. The ability of arrays to support this kind of operation is one of their prime virtues.

Program 3.8 Closest-point computation

This program illustrates the use of an array of structures, and is representative of the typical situation where we save items in an array to process them later, during some computation. It counts the number of pairs of N randomly generated points in the unit square that can be connected by a straight line of length less than d , using the data type for points described in Section 3.1. The running time is $O(N^2)$, so this program cannot be used for huge N . Program 3.20 provides a faster solution.

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "Point.h"
float randFloat()
{ return 1.0*rand()/RAND_MAX; }
main(int argc, char *argv[])
{ float d = atof(argv[2]);
  int i, j, cnt = 0, N = atoi(argv[1]);
  point *a = malloc(N*(sizeof(*a)));
  for (i = 0; i < N; i++)
    { a[i].x = randFloat(); a[i].y = randFloat(); }
  for (i = 0; i < N; i++)
    for (j = i+1; j < N; j++)
      if (distance(a[i], a[j]) < d) cnt++;
  printf("%d edges shorter than %f\n", cnt, d);
}
```

Programs 3.5 and 3.7 both compute array indices from the data at hand. In a sense, when we use a computed value to access an array of size N , we are taking N possibilities into account with just a single operation. This gain in efficiency is compelling when we can realize it, and we shall be encountering algorithms throughout the book that make use of arrays in this way.

We use arrays to organize all different manner of types of objects, not just integers. In C, we can declare arrays of any built-in or user-defined type (i.e., compound objects declared as structures). Program 3.8 illustrates the use of an array of structures for points in the plane using the structure definition that we considered in Section 3.1.

This program also illustrates a common use of arrays: to save data away so that they can be quickly accessed in an organized manner in some computation. Incidentally, Program 3.8 is also interesting as a prototypical quadratic algorithm, which checks all pairs of a set of N data items, and therefore takes time proportional to N^2 . In this book, we look for improvements whenever we see such an algorithm, because its use becomes infeasible as N grows. In this case, we shall see how to use a compound data structure to perform this computation in linear time, in Section 3.7.

We can create compound types of arbitrary complexity in a similar manner: We can have not just arrays of structs, but also arrays of arrays, or structs containing arrays. We will consider these different options in detail in Section 3.7. Before doing so, however, we will examine *linked lists*, which serve as the primary alternative to arrays for organizing collections of objects.

Exercises

- ▷ 3.11 Suppose that `a` is declared as `int a[99]`. Give the contents of the array after the following two statements are executed:


```
for (i = 0; i < 99; i++) a[i] = 98-i;
for (i = 0; i < 99; i++) a[i] = a[a[i]];
```
- 3.12 Modify our implementation of the sieve of Eratosthenes (Program 3.5) to use an array of (i) chars; and (ii) bits. Determine the effects of these changes on the amount of space and time used by the program.
- ▷ 3.13 Use the sieve of Eratosthenes to determine the number of primes less than N , for $N = 10^3$, 10^4 , 10^5 , and 10^6 .
- 3.14 Use the sieve of Eratosthenes to draw a plot of N versus the number of primes less than N for N between 1 and 1000.
- 3.15 Empirically determine the effect of removing the test `if (a[i])` that guards the inner loop of Program 3.5, for $N = 10^3$, 10^4 , 10^5 , and 10^6 .
- 3.16 Analyze Program 3.5 to explain the effect that you observed in Exercise 3.15.
- ▷ 3.17 Write a program that counts the number of different integers less than 1000 that appear in an input stream.
- 3.18 Write a program that determines empirically the number of random positive integers less than 1000 that you can expect to generate before getting a repeated value.
- 3.19 Write a program that determines empirically the number of random positive integers less than 1000 that you can expect to generate before getting each value at least once.

- 3.20 Modify Program 3.7 to simulate a situation where the coin turns up heads with probability p . Run 1000 trials for an experiment with 32 flips with $p = 1/6$ to get output that you can compare with Figure 3.2.
- 3.21 Modify Program 3.7 to simulate a situation where the coin turns up heads with probability λ/N . Run 1000 trials for an experiment with 32 flips to get output that you can compare with Figure 3.2. This distribution is the classical *Poisson* distribution.
- 3.22 Modify Program 3.8 to print out the coordinates of the closest pair of points.
 - 3.23 Modify Program 3.8 to perform the same computation in d dimensions.

3.3 Linked Lists

When our primary interest is to go through a collection of items sequentially, one by one, we can organize the items as a *linked list*: a basic data structure where each item contains the information that we need to get to the next item. The primary advantage of linked lists over arrays is that the links provide us with the capability to rearrange the items efficiently. This flexibility is gained at the expense of quick access to any arbitrary item in the list, because the only way to get to an item in the list is to follow links, one node to the next. There are a number of ways to organize linked lists, all starting with the following basic definition.

Definition 3.2 A linked list is a set of items where each item is part of a node that also contains a link to a node.

We define nodes in terms of references to nodes, so linked lists are sometimes referred to as *self-referent* structures. Moreover, although a node's link usually refers to a different node, it could refer to the node itself, so linked lists can also be *cyclic* structures. The implications of these two facts will become apparent as we begin to consider concrete representations and applications of linked lists.

Normally, we think of linked lists as implementing a sequential arrangement of a set of items: Starting at a given node, we consider its item to be first in the sequence. Then, we follow its link to another node, which gives us an item that we consider to be second in the sequence, and so forth. In principle, the list could be cyclic and the sequence could seem infinite, but we most often work with lists that

correspond to a simple sequential arrangement of a finite set of items, adopting one of the following conventions for the link in the final node:

- It is a *null link* that points to no node.
- It refers to a *dummy node* that contains no item.
- It refers back to the first node, making the list a *circular list*.

In each case, following links from the first node to the final one defines a sequential arrangement of items. Arrays define a sequential ordering of items as well; in an array, however, the sequential organization is provided implicitly, by the position in the array. (Arrays also support arbitrary access by index, which lists do not.)

We first consider nodes with precisely one link, and, in most applications, we work with one-dimensional lists where all nodes except possibly the first one and the final one each have precisely one link referring to them. This corresponds to the simplest situation, which is also the one that interests us most, where linked lists correspond to finite sequences of items. We will consider more complicated situations in due course.

Linked lists are defined as a primitive in some programming environments, but not in C. However, the basic building blocks that we discussed in Section 3.1 are well suited to implementing linked lists. Specifically, we use pointers for links and structures for nodes. The typedef declaration gives us a way to refer to links and nodes, as follows:

```
typedef struct node *link;  
struct node { Item item; link next; };
```

which is nothing more than C code for Definition 3.2. Links are pointers to nodes, and nodes consist of items and links. We assume that another part of the program uses typedef or some other mechanism to allow us to declare variables of type `Item`. We shall see more complicated representations in Chapter 4 that provide more flexibility and allow more efficient implementations of certain operations, but this simple representation will suffice for us to consider the fundamentals of list processing. We use similar conventions for linked structures throughout the book.

Memory allocation is a central consideration in the effective use of linked lists. Although we have defined a single structure (`struct node`), it is important to remember that we will have many instances of

this structure, one for each node that we want to use. Generally, we do not know the number of nodes that we will need until our program is executing, and various parts of our programs might have similar calls on the available memory, so we make use of system programs to keep track of our memory usage. To begin, whenever we want to use a new node, we need to create an instance of a node structure and to reserve memory for it—for example, we typically write code such as

```
link x = malloc(sizeof *x);
```

to direct the `malloc` function from `stdlib.h` and the `sizeof` operator to reserve enough memory for a node and to return a pointer to it in `x`. (This line of code does not refer directly to node, but a link can only refer to a node, so `sizeof` and `malloc` have the information that they need.) In Section 3.5, we shall consider the memory-allocation process in more detail. For the moment, for simplicity, we regard this line of code as a C idiom for creating new nodes. Indeed, our use of `malloc` is structured in this way throughout this book.

Now, once a list node is created, how do we refer to the information it comprises—its item and its link? We have already seen the basic operations that we need for this task: We simply dereference the pointer, then use the structure member names—the item in the node referenced by link `x` (which is of type `Item`) is `(*x).item` and the link (which is of type `link`) is `(*x).link`. These operations are so heavily used, however, that C provides the shorthand `x->item` and `x->link`, which are equivalent forms. Also, we so often need to use the phrase “the node referenced by link `x`” that we simply say “node `x`”—the link *does* name the node.

The correspondence between links and C pointers is essential, but we must bear in mind that the former is an abstraction and the latter a concrete representation. For example, we can also represent links with array indices, as we shall see at the end of this section.

Figures 3.3 and 3.4 show the two fundamental operations that we perform on linked lists. We can *delete* any item from a linked list, to make it shrink by 1 in length; and we can *insert* an item into a linked list at any point, to make it grow by 1 in length. For simplicity, we assume in these figures that the lists are circular and never become empty. We will consider null links, dummy nodes, and empty lists in Section 3.4. As shown in the figures, insertion and deletion each

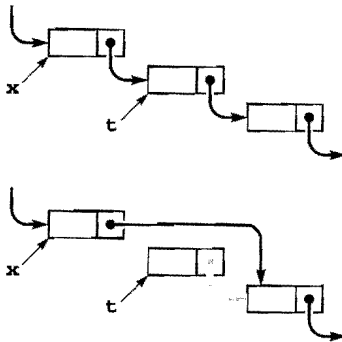


Figure 3.3
Linked-list deletion

To delete, or remove, the node following a given node `x` from a linked list, we set `t` to point to the node to be removed, then change `x`'s link to point to `t->next`. The pointer `t` can be used to refer to the removed node (to return it to a free list, for example). Although its link still points into the list, we generally do not use such a link after removing the node from the list, except perhaps to inform the system, via `free`, that its memory can be reclaimed.

require just two statements in C. To delete the node following node x , we use the statements

```
t = x->next; x->next = t->next;
```

or simply

```
x->next = x->next->next;
```

To insert node t into a list at a position following node x , we use the statements

```
t->next = x->next; x->next = t;
```

The simplicity of insertion and deletion is the *raison d'être* of linked lists. The corresponding operations are unnatural and inconvenient in arrays, because they require moving all of the array's contents following the affected item.

By contrast, linked lists are *not* well suited for the *find the k th item* (find an item given its index) operation that characterizes efficient access in arrays. In an array, we find the k th item simply by accessing $a[k]$; in a list, we have to traverse k links. Another operation that is unnatural on singly linked lists is “find the item *before* a given item.”

When we remove a node from a linked list using $x->next = x->next->next$, we may never be able to access it again. For small programs such as the examples we consider at first, this is no special concern, but we generally regard it as good programming practice to use the function `free`, which is the counterpart to `malloc`, for any node that we no longer wish to use. Specifically, the sequence of instructions

```
t = x->next; x->next = t->next; free(t);
```

not only removes t from our list but also informs the system that the memory it occupies may be used for some other purpose. We pay particular attention to `free` when we have large list objects, or large numbers of them, but we will ignore it until Section 3.5, so that we may focus on appreciating the benefits of linked structures.

We will see many examples of applications of these and other basic operations on linked lists in later chapters. Since the operations involve only a few statements, we often manipulate the lists directly rather than defining functions for inserting, deleting, and so forth. As an example, we consider next a program for solving the *Josephus problem* in the spirit of the sieve of Eratosthenes.

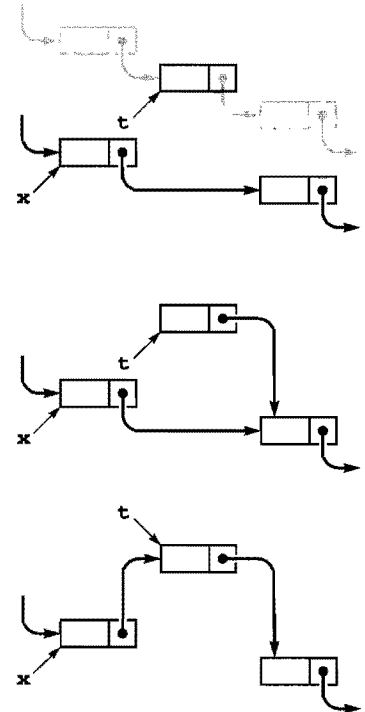


Figure 3.4
Linked-list insertion

To insert a given node t into a linked list at a position following another given node x (top), we set $t->next$ to $x->next$ (center), then set $x->next$ to t (bottom).

Program 3.9 Circular list example (Josephus problem)

To represent people arranged in a circle, we build a circular linked list, with a link from each person to the person on the left in the circle. The integer i represents the i th person in the circle. After building a one-node circular list for 1, we insert 2 through N after that node, resulting in a circle with 1 through N , leaving x pointing to N . Then, we skip $M - 1$ nodes, beginning with 1, and set the link of the $(M - 1)$ st to skip the M th, continuing until only one node is left.

```
#include <stdlib.h>
typedef struct node* link;
struct node { int item; link next; };
main(int argc, char *argv[])
{ int i, N = atoi(argv[1]), M = atoi(argv[2]);
  link t = malloc(sizeof *t), x = t;
  t->item = 1; t->next = t;
  for (i = 2; i <= N; i++)
  {
    x = (x->next = malloc(sizeof *x));
    x->item = i; x->next = t;
  }
  while (x != x->next)
  {
    for (i = 1; i < M; i++) x = x->next;
    x->next = x->next->next; N--;
  }
  printf("%d\n", x->item);
}
```

We imagine that N people have decided to elect a leader by arranging themselves in a circle and eliminating every M th person around the circle, closing ranks as each person drops out. The problem is to find out which person will be the last one remaining (a mathematically inclined potential leader will figure out ahead of time which position in the circle to take). The identity of the elected leader is a function of N and M that we refer to as the *Josephus function*. More generally, we may wish to know the order in which the people are eliminated. For example, as shown in Figure 3.5, if $N = 9$ and $M = 5$, the people

are eliminated in the order 5 1 7 4 3 6 9 2, and 8 is the leader chosen. Program 3.9 reads in N and M and prints out this ordering.

Program 3.9 uses a *circular* linked list to simulate the election process directly. First, we build the list for 1 to N : We build a circular list consisting of a single node for person 1, then insert the nodes for people 2 through N , in that order, following that node in the list, using the insertion code illustrated in Figure 3.4. Then, we proceed through the list, counting through $M - 1$ items, deleting the next one using the code illustrated in Figure 3.3, and continuing until only one node is left (which then points to itself).

The sieve of Eratosthenes and the Josephus problem clearly illustrate the distinction between using arrays and using linked lists to represent a sequentially organized collection of objects. Using a linked list instead of an array for the sieve of Eratosthenes would be costly because the algorithm's efficiency depends on being able to access any array position quickly, and using an array instead of a linked list for the Josephus problem would be costly because the algorithm's efficiency depends on the ability to delete items quickly. When we choose a data structure, we *must* be aware of the effects of that choice upon the efficiency of the algorithms that will process the data. This interplay between data structures and algorithms is at the heart of the design process and is a recurring theme throughout this book.

In C, pointers provide a direct and convenient concrete realization of the abstract concept of a linked list, but the essential value of the abstraction does not depend on any particular implementation. For example, Figure 3.6 shows how we could use arrays of integers to implement the linked list for the Josephus problem. That is, we can implement linked lists using array indices, instead of pointers. Linked lists are thus useful even in the simplest of programming environments. Linked lists were useful well before pointer constructs were available in high-level languages such as C. Even in modern systems, simple array-based implementations are sometimes convenient.

Exercises

- ▷ 3.24 Write a function that returns the number of nodes on a circular list, given a pointer to one of the nodes on the list.
- 3.25 Write a code fragment that determines the number of nodes that are between the nodes referenced by two given pointers x and t to nodes on a circular list.

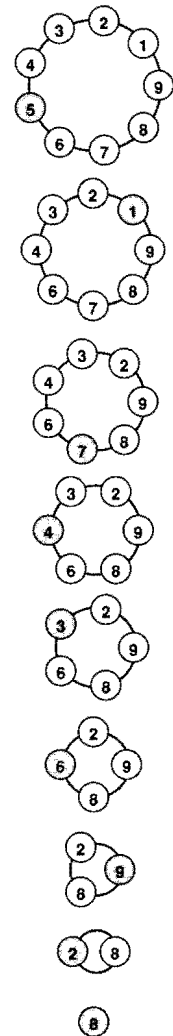


Figure 3.5
Example of Josephus election

This diagram shows the result of a Josephus-style election, where the group stands in a circle, then counts around the circle, eliminating every fifth person and closing the circle.

	0	1	2	3	4	5	6	7	8
item	1	2	3	4	5	6	7	8	9
next	1	2	3	4	5	6	7	8	0
5	1	2	3	4	5	6	7	8	9
	1	2	3	5	5	6	7	8	0
1	1	2	3	4	5	6	7	8	9
	1	2	3	5	5	6	7	8	1
7	1	2	3	4	5	6	7	8	9
	1	2	3	5	5	7	7	8	1
4	1	2	3	4	5	6	7	8	9
	1	2	5	5	5	7	7	8	1
3	1	2	3	4	5	6	7	8	9
	1	5	5	5	5	7	7	8	1
6	1	2	3	4	5	6	7	8	9
	1	7	5	5	5	7	7	8	1
9	1	2	3	4	5	6	7	8	9
	1	7	5	5	5	7	7	1	1
2	1	2	3	4	5	6	7	8	9
	1	7	5	5	5	7	7	7	1

Figure 3.6
Array representation of a
linked list

This sequence shows the linked list for the Josephus problem (see Figure 3.5), implemented with array indices instead of pointers. The index of the item following the item with index 0 in the list is `next[0]`, and so forth. Initially (top three rows), the item for person i has index $i-1$, and we form a circular list by setting `next[i]` to $i+1$ for i from 0 to 8 and `next[8]` to 0. To simulate the Josephus-election process, we change the links (`next` array entries) but do not move the items. Each pair of lines shows the result of moving through the list four times with $x = \text{next}[x]$, then deleting the fifth item (displayed at the left) by setting `next[x]` to `next[next[x]]`.

3.26 Write a code fragment that, given pointers x and t to two disjoint circular lists, inserts the list pointed to by t into the list pointed to by x , at the point following x .

• 3.27 Given pointers x and t to nodes on a circular list, write a code fragment that moves the node following t to the position following the node following x on the list.

3.28 When building the list, Program 3.9 sets twice as many link values as it needs to because it maintains a circular list after each node is inserted. Modify the program to build the circular list without doing this extra work.

3.29 Give the running time of Program 3.9, within a constant factor, as a function of M and N .

3.30 Use Program 3.9 to determine the value of the Josephus function for $M = 2, 3, 5, 10$, and $N = 10^3, 10^4, 10^5$, and 10^6 .

3.31 Use Program 3.9 to plot the Josephus function versus N for $M = 10$ and N from 2 to 1000.

○ 3.32 Redo the table in Figure 3.6, beginning with item i initially at position $N-i$ in the array.

3.33 Develop a version of Program 3.9 that uses an array of indices to implement the linked list (see Figure 3.6).

3.4 Elementary List Processing

Linked lists bring us into a world of computing that is markedly different from that of arrays and structures. With arrays and structures, we save an item in memory and later refer to it by name (or by index) in much the same manner as we might put a piece of information in a file drawer or an address book; with linked lists, the manner in which we save information makes it more difficult to access but easier to rearrange. Working with data that are organized in linked lists is called *list processing*.

When we use arrays, we are susceptible to program bugs involving out-of-bounds array accesses. The most common bug that we encounter when using linked lists is a similar bug where we reference an undefined pointer. Another common mistake is to use a pointer that we have changed unknowingly. One reason that this problem arises is that we may have multiple pointers to the same node without necessarily realizing that that is the case. Program 3.9 avoids several such problems by using a circular list that is never empty, so that each

link always refers to a well-defined node, and each link can also be interpreted as referring to the list.

Developing correct and efficient code for list-processing applications is an acquired programming skill that requires practice and patience to develop. In this section, we consider examples and exercises that will increase our comfort with working with list-processing code. We shall see numerous other examples throughout the book, because linked structures are at the heart of some of our most successful algorithms.

As mentioned in Section 3.3, we use a number of different conventions for the first and final pointers in a list. We consider some of them in this section, even though we adopt the policy of reserving the term *linked list* to describe the simplest situation.

Definition 3.3 *A linked list is either a null link or a link to a node that contains an item and a link to a linked list.*

This definition is more restrictive than Definition 3.2, but it corresponds more closely to the mental model that we have when we write list-processing code. Rather than exclude all the other various conventions by using only this definition, and rather than provide specific definitions corresponding to each convention, we let both stand, with the understanding that it will be clear from the context which type of linked list we are using.

One of the most common operations that we perform on lists is to *traverse* them: We scan through the items on the list sequentially, performing some operation on each. For example, if x is a pointer to the first node of a list, the final node has a null pointer, and $visit$ is a function that takes an item as an argument, then we might write

```
for (t = x; t != NULL; t = t->next) visit(t->item);
```

to traverse the list. This loop (or its equivalent `while` form) is as ubiquitous in list-processing programs as is the corresponding

```
for (i = 0; i < N; i++)
```

in array-processing programs.

Program 3.10 is an implementation of a simple list-processing task, reversing the order of the nodes on a list. It takes a linked list as an argument, and returns a linked list comprising the same nodes, but with the order reversed. Figure 3.7 shows the change that the

Program 3.10 List reversal

This function reverses the links in a list, returning a pointer to the final node, which then points to the next-to-final node, and so forth, with the link in the first node of the original list set to NULL. To accomplish this task, we need to maintain links to three consecutive nodes in the list.

```
link reverse(link x)
{ link t, y = x, r = NULL;
  while (y != NULL)
    { t = y->next; y->next = r; r = y; y = t; }
  return r;
}
```

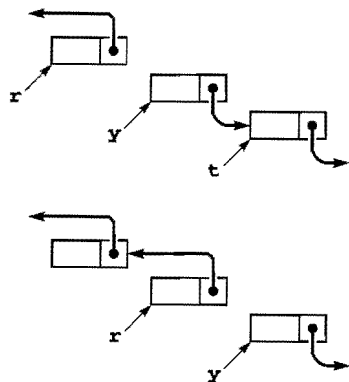


Figure 3.7
List reversal

To reverse the order of a list, we maintain a pointer *r* to the portion of the list already processed, and a pointer *y* to the portion of the list not yet seen. This diagram shows how the pointers change for each node in the list. We save a pointer to the node following *y* in *t*, change *y*'s link to point to *r*, and then move *r* to *y* and *y* to *t*.

function makes for each node in its main loop. Such a diagram makes it easier for us to check each statement of the program to be sure that the code changes the links as intended, and programmers typically use these diagrams to understand the operation of list-processing implementations.

Program 3.11 is an implementation of another list-processing task: rearranging the nodes of a list to put their items in sorted order. It generates N random integers, puts them into a list in the order that they were generated, rearranges the nodes to put their items in sorted order, and prints out the sorted sequence. As we discuss in Chapter 6, the expected running time of this program is proportional to N^2 , so the program is not useful for large N . Beyond this observation, we defer discussing the sort aspect of this program to Chapter 6, because we shall see a great many methods for sorting in Chapters 6 through 10. Our purpose now is to present the implementation as an example of a list-processing application.

The lists in Program 3.11 illustrate another commonly used convention: We maintain a dummy node called a *head node* at the beginning of each list. We ignore the item field in a list's head node, but maintain its link as the pointer to the node containing the first item in the list. The program uses two lists: one to collect the random input in the first loop, and the other to collect the sorted output in the second loop. Figure 3.8 diagrams the changes that Program 3.11 makes during one iteration of its main loop. We take the next node

Program 3.11 List insertion sort

This code generates N random integers between 0 and 999, builds a linked list with one number per node (first for loop), and then rearranges the nodes so that the numbers appear in order when we traverse the list (second for loop). To accomplish the sort, we maintain two lists, an input (unsorted) list and an output (sorted) list. On each iteration of the loop, we remove a node from the input and insert it into position in the output. The code is simplified by the use of head nodes for each list, that contain the links to the first nodes on the lists. For example, without the head node, the case where the node to be inserted into the output list goes at the beginning would involve extra code.

```

struct node heada, headb;
link t, u, x, a = &heada, b;
for (i = 0, t = a; i < N; i++)
{
    t->next = malloc(sizeof *t);
    t = t->next; t->next = NULL;
    t->item = rand() % 1000;
}
b = &headb; b->next = NULL;
for (t = a->next; t != NULL; t = u)
{
    u = t->next;
    for (x = b; x->next != NULL; x = x->next)
        if (x->next->item > t->item) break;
    t->next = x->next; x->next = t;
}

```

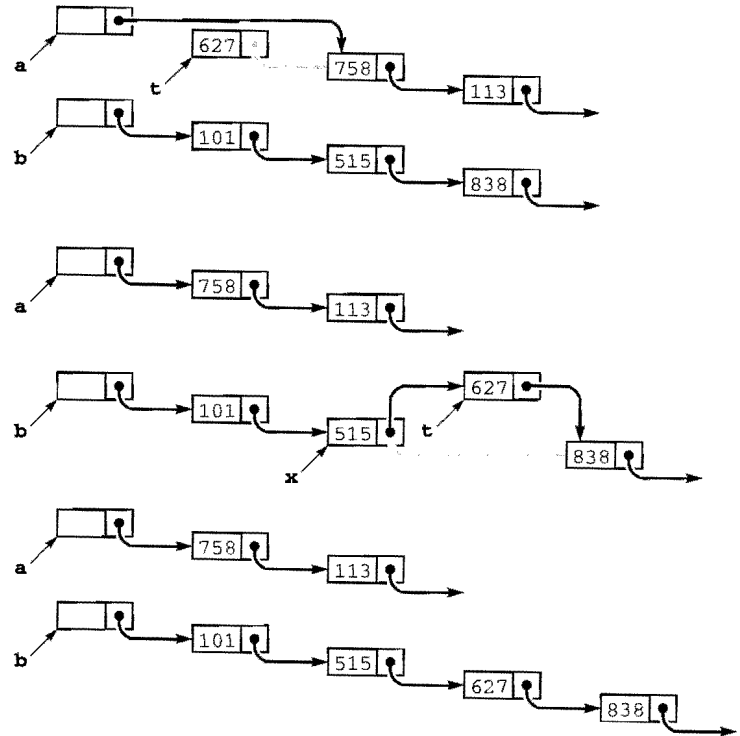
off the input list, find where it belongs in the output list, and link it into position.

The primary reason to use the head node at the beginning becomes clear when we consider the process of adding the *first* node to the sorted list. This node is the one in the input list with the smallest item, and it could be anywhere on the list. We have three options:

- Duplicate the for loop that finds the smallest item and set up a one-node list in the same manner as in Program 3.9.
- Test whether the output list is empty every time that we wish to insert a node.

Figure 3.8
Linked-list sort

This diagram depicts one step in transforming an unordered linked list (pointed to by *a*) into an ordered one (pointed to by *b*), using insertion sort. We take the first node of the unordered list, keeping a pointer to it in *t* (top). Then, we search through *b* to find the first node *x* with *x*->next->item > *t*->item (or *x*->next = NULL), and insert *t* into the list following *x* (center). These operations reduce the length of *a* by one node, and increase the length of *b* by one node, keeping *b* in order (bottom). Iterating, we eventually exhaust *a* and have the nodes in order in *b*.



- Use a dummy head node whose link points to the first node on the list, as in the given implementation.

The first option is inelegant and requires extra code; the second is also inelegant and requires extra time.

The use of a head node does incur some cost (the extra node), and we can avoid the head node in many common applications. For example, we can also view Program 3.10 as having an input list (the original list) and an output list (the reversed list), but we do not need to use a head node in that program because all insertions into the output list are at the beginning. We shall see still other applications that are more simply coded when we use a dummy node, rather than a null link, at the *tail* of the list. There are no hard-and-fast rules about whether or not to use dummy nodes—the choice is a matter of style combined with an understanding of effects on performance. Good programmers

Table 3.1 Head and tail conventions in linked lists

This table gives implementations of basic list-processing operations with five commonly used conventions. This type of code is used in simple applications where the list-processing code is inline.

Circular, never empty

```

first insert: head->next = head;
insert t after x: t->next = x->next; x->next = t;
delete after x: x->next = x->next->next;
traversal loop: t = head;
                  do { ... t = t->next; } while (t != head);
test if one item: if (head->next == head)

```

Head pointer, null tail

```

initialize: head = NULL;
insert t after x: if (x == NULL) { head = t; head->next = NULL; }
                  else { t->next = x->next; x->next = t; }
delete after x: t = x->next; x->next = t->next;
traversal loop: for (t = head; t != NULL; t = t->next)
test if empty: if (head == NULL)

```

Dummy head node, null tail

```

initialize: head = malloc(sizeof *head);
             head->next = NULL;
insert t after x: t->next = x->next; x->next = t;
delete after x: t = x->next; x->next = t->next;
traversal loop: for (t = head->next; t != NULL; t = t->next)
test if empty: if (head->next == NULL)

```

Dummy head and tail nodes

```

initialize: head = malloc(sizeof *head);
             z = malloc(sizeof *z);
             head->next = z; z->next = z;
insert t after x: t->next = x->next; x->next = t;
delete after x: x->next = x->next->next;
traversal loop: for (t = head->next; t != z; t = t->next)
test if empty: if (head->next == z)

```

Program 3.12 List-processing interface

This code, which might be kept in an interface file `list.h`, specifies the types of nodes and links, and declares some of the operations that we might want to perform on them. We declare our own functions for allocating and freeing memory for list nodes. The function `initNodes` is for the convenience of the implementation. The `typedef` for `Node` and the functions `Next` and `Item` allow clients to use lists without dependence upon implementation details.

```
typedef struct node* link;
struct node { itemType item; link next; };
typedef link Node;
void initNodes(int);
link newNode(int);
void freeNode(link);
void insertNext(link, link);
link deleteNext(link);
link Next(link);
int Item(link);
```

enjoy the challenge of picking the convention that most simplifies the task at hand. We shall see several such tradeoffs throughout this book.

For reference, a number of options for linked-list conventions are laid out in Table 3.1; others are discussed in the exercises. In all the cases in Table 3.1, we use a pointer `head` to refer to the list, and we maintain a consistent stance that our program manages links to nodes, using the given code for various operations. Allocating and freeing memory for nodes and filling them with information is the same for all the conventions. Robust functions implementing the same operations would have extra code to check for error conditions. The purpose of the table is to expose similarities and differences among the various options.

Another important situation in which it is sometimes convenient to use head nodes occurs when we want to pass pointers to lists as arguments to functions that may modify the list, in the same way that we do for arrays. Using a head node allows the function to accept or return an empty list. If we do not have a head node, we need a mechanism for the function to inform the calling program when